Python Programming

Unit: 1

Lecture 1 Introduction to Pythonand Python variables.

1.1 Introduction to Python: Python is a popular programming language. It wascreated by Guido van Rossum, and released in 1991.

It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

What can Python do?

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modifyfiles.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-readysoftware development.

1.2 Python Variables: Python Variable is containers that store values. Python is not "statically typed". We do not need to declare variables before using them or declare their type. A variable is created the moment we first assign a value to it. A Python variable is a name given to a memory location. It is the basic unit of storage in a program. An Example of a Variable in Python is a representational name that serves as a pointer to an object. Once an object is assigned to a variable, it can be referred to by that name. In layman's terms, we can say that Variable in Python is containers that store values.

Example:
```
Var = "Ram"
print(Var)
```

Output: Ram

Notes:

- The value stored in a variable can be changed during program execution.
- A Variables in Python is only a name given to a memory location, all theoperations done on the variable effects that memory location.

1.2.1 Rules for Python variables:

- A Python variable name must start with a letter or the underscorecharacter.
- A Python variable name cannot start with a number.
- A Python variable name can only contain alpha-numeric characters andunderscores (A-z, 0-9, and _).
- Variable in Python names are case-sensitive (name, Name, and NAME are three different variables).
- The reserved words (keywords) in Python cannot be used to name thevariable in Python.

Lecture 2. Python basic Operators and its Types

1.3 Python basic Operators: In Python programming, Operators in general are used to perform operations on values and variables. These are standard symbols used for the purpose of logical and arithmetic operations.

- OPERATORS: These are the special symbols. Eg- + , * , /, etc.
- OPERAND: It is the value on which the operator is applied.

Types of Operators in Python

1. Arithmetic Operators
2. Comparison Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Identity Operators and Membership Operators

1.3.1 Arithmetic Operators in Python:

Python Arithmetic operators are used to perform basic mathematicaloperations like addition, subtraction, multiplication, and division.

In Python 3.x the result of division is a floating-point while in Python 2.xdivision of 2 integers was an integer. To obtain an integer result in Python 3.x floored (// integer) is used.

Operator	Description	Syntax
+	Addition: adds two operands	x + y
–	Subtraction: subtracts two operands	x – y
*	Multiplication: multiplies two operands	x * y
/	Division (float): divides the first operand by the second	x / y
//	Division (floor): divides the first operand by the second	x // y
%	Modulus: returns the remainder when the first operandis divided by the second	x % y

**	Power: Returns first raised to power second	x ** y

Table 1.1-Arithmetic Operators in Python

Division Operators-

Division Operators allow you to divide two numbers and return a quotient, i.e., the first number or number at the left is divided by the second numberor number at the right and returns the quotient.

There are two types of division operators:

1. Float division
2. Floor division

Float division

The quotient returned by this operator is always a float number, no matter iftwo numbers are integers. For example:

```
# python program to demonstrate the use of "/"

print(5/5)

print(10/2)

print(-10/2)

print(20.0/2)
```

Output:

```
1.0
5.0
-5.0
10.0
```

Integer division (Floor division)

The quotient returned by this operator is dependent on the argument being passed. If any of the numbers is float, it returns output in float. It is also known as Floor division because, if any number is negative, then the output will be floored. For example:

```
# python program to demonstrate the use of "//"

print(10//3)

print (-5//2)

print (5.0//2)

print (-5.0//2)
```

Output:

3

-3

2.0

-3.0

Precedence of Arithmetic Operators in Python

The precedence of Arithmetic Operators in python is as follows:

1. P – Parentheses
2. E – Exponentiation
3. M – Multiplication (Multiplication and division have the same precedence)
4. D – Division
5. A – Addition (Addition and subtraction have the same precedence)
6. S – Subtraction

The modulus operator helps us extract the last digit/s of a number. Forexample:

- x % 10 -> yields the last digit
- x % 100 -> yield last two digits

Arithmetic Operators With Addition, Subtraction, Multiplication, Modulo and Power
Here is an example showing how different Arithmetic Operators in Pythonwork:

```
# Examples of Arithmetic Operator

a = 9

b = 4

# Addition of numbers

add = a + b

# Subtraction of numbers
```

```
sub = a – b

# Multiplication of number
mul = a * b

# Modulo of both number

mod = a % b

#  Power

 p = a ** b

# print results

print(add)

print(sub)

print(mul)
print(mod)
print(p)
```

13

5

36

1

6561

1.3.2 Comparison Operators in Python

In Python Comparison of Relational operators compares the values. It either returns **True** or **False** according to the condition.

Operator	**Description**	**Syntax**
>	Greater than: True if the left operand is greater than theright	x > y
<	Less than: True if the left operand is less than the right	x < y
==	Equal to: True if both operands are equal	x == y
!=	Not equal to – True if operands are not equal	x != y
>=	Greater than or equal to True if the left operand isgreater than or equal to the right	x >= y

<=	Less than or equal to True if the left operand is less thanor equal to the right	x <= y

Table 1.2 Comparison Operators

Note = is an assignment operator and == comparison operator.

Precedence of Comparison Operators in Python

In python, the comparison operators have lower precedence than the arithmetic operators. All the operators within comparison operators havesame precedence order.

Example of Comparison Operators in Python

Let's see an example of Comparison Operators in Python.

```
# Examples of Relational Operators

a = 13

b = 33
# a > b is False

print(a > b)

# a < b is True

print(a < b)

# a == b is False

print(a == b)

# a != b is True

print(a != b)


 # a >= b is False


 print(a >= b)

 # a <= b is True

 print(a <= b)
```

Output

```
False
True
False
True
False
True
```

1.3.3 Logical Operators in Python

Python Logical operators perform **Logical AND**, **Logical OR**, and **LogicalNOT** operations. It is used to combine conditional statements.

Operator	Description	Syntax
And	Logical AND: True if both the operands are true	x and y
Or	Logical OR: True if either of the operands is true	x or y
Not	Logical NOT: True if the operand is false	not x

Table 1.3 Logical Operators

Precedence of Logical Operators in Python

The precedence of Logical Operators in python is as follows:

1. Logical not
2. logical and
3. logical or

Example of Logical Operators in Python

The following code shows how to implement Logical Operators in Python:

```
# Examples of Logical Operator

a = True

b = False

# Print a and b is False

print(a and b)

# Print a or b is True

print(a or b)

# Print not a is False

print(not a)
```

Output

```
False
True
False
```

1.3.4 Bitwise Operators in Python

Python Bitwise operators act on bits and perform bit-by-bit operations.These are used to operate on binary numbers.

Operator	Description	Syntax
&	Bitwise AND	x & y
|	Bitwise OR	x | y
~	Bitwise NOT	~x
^	Bitwise XOR	x ^ y
>>	Bitwise right shift	x>>
<<	Bitwise left shift	x<<

Table 1.4 Bitwise Operators

Precedence of Bitwise Operators in Python

The precedence of Bitwise Operators in python is as follows:

1. Bitwise NOT
2. Bitwise Shift
3. Bitwise AND
4. Bitwise XOR
5. Bitwise OR

Bitwise Operators in Python Here is an example showing how Bitwise Operators in Python work:

```
# Examples of Bitwise operators

a = 10

b = 4

# Print bitwise AND operation

print(a  & b)

# Print bitwise OR operation

print(a | b)


# Print bitwise NOT operation

print(~a)
```

```
# print bitwise XOR operation

print(a ^ b)

# print bitwise right shift operation

print(a >> 2)

# print bitwise left shift operation

print(a << 2)
```

Output

```
0
14
-11
14
2
40
```

1.3.5 Assignment Operators in Python

Operator	Description	Syntax
=	Assign the value of the right side of the expression to the left side operand	x = y + z
+=	Add AND: Add right-side operand with left-sideoperand and then assign to left operand	a+=b a=a+b
-=	Subtract AND: Subtract right operand from leftoperand and then assign to left operand	a-=b a=a-b
*=	Multiply AND: Multiply right operand with leftoperand and then assign to left operand	a*=b a=a*b
/=	Divide AND: Divide left operand with rightoperand and then assign to left operand	a/=b a=a/b
%=	Modulus AND: Takes modulus using left andright operands and assign the result to left operator	a%=b a=a%b

//=	Divide(floor) AND: Divide left operand with right operand and then assign the value(floor) to left operand	a//=b a=a//b
**=	Exponent AND: Calculate exponent(raise power) value using operands and assign value to left operand	a**=b a=a**b
&=	Performs Bitwise AND on operands and assignvalue to left operand	a&=b a=a&b
|=	Performs Bitwise OR on operands and assignvalue to left operand	a|=b a=a|b
^=	Performs Bitwise xOR on operands and assignvalue to left operand	a^=b a=a^b
>>=	Performs Bitwise right shift on operands andassign value to left operand	a>>=b a=a>>b
<<=	Performs Bitwise left shift on operands and assign value to left operand	a <<= b a=a << b

Table 1.5 Assignment Operators

Python Assignment operators are used to assign values to the variables.

Assignment Operators in Python

Let's see an example of Assignment Operators in Python.

```
# Examples of Assignment Operators

a = 10

# Assign value

b = a

print(b)

# Add and assign value

 b += a

print(b)

# Subtract and assign value

b -= a

print(b)
```

```
# multiply and assign

b *= a

print(b)

# bitwise left shift operator

b <<= a

print(b)

# bitwise right shift operator

b >>= a

print(b)
```

Output

```
10
20
10
100
102400
0.09765625
```

1.3.6 Identity Operators in Python

In Python, **is** and **is not** are the identity operators both are used to check if two values are located on the same part of the memory. Two variables thatare equal do not imply that they are identical.

is True if the operands are identical

is not True if the operands are not identical

Example Identity Operators in Python

Let's see an example of Identity Operators in Python.

```
a = 10
b = 20
c = a


print(a is not b)

print(a is c)
```

Output

True

True

1.3.7 Membership Operators in Python

In Python, **in** and **not in** are the membership operators that are used to testwhether a value or variable is in a sequence.

in True if value is found in the sequence

not in True if value is not found in the sequence

Examples of Membership Operators in Python

The following code shows how to implement Membership Operators inPython:

```
# Python program to illustrate ‘in’ and ‘not in' operator

x = 24
y = 20

list = [10, 20, 30, 40, 50]

if (x not in list):

    print("x is NOT present in given list")

else:

    print("x is present in given list")

 if (y in list):

    print("y is present in given list")

else:

    print("y is NOT present in given list")
```

Output

x is NOT present in given list

y is present in given list

1.3.8 Ternary Operator in Python

in Python, Ternary operators also known as conditional expressions are operators that evaluate something based on a condition being true or false. It was added to Python in version 2.5.
It simply allows testing a condition in a **single line** replacing the multiline if-else making the code compact.

Syntax : *[on_true] if [expression] else [on_false]*

Examples of Ternary Operator in Python

Here is a simple example of Ternary Operator in Python.

```
# Program to demonstrate ternary operator
a = 10
b = 20

# python ternary operator
min = "a is minimum" if a < b else "b is minimum"

print(min)
```

Output:

a is minimum

1.4 Precedence and Associativity of Operators in Python

In Python, Operator precedence and associativity determine the priorities ofthe operator.

1.4.1 Operator Precedence in Python

This is used in an expression with more than one operator with differentprecedence to determine which operation to perform first.

Let’s see an example of how Operator Precedence in Python works:

```
# Examples of Operator Precedence
# Precedence of '+' & '*'

expr = 10 + 20 * 30

print(expr)

# Precedence of 'or' & 'and'

name = "Alex"

age = 0

if name == "Alex" or name == "John" and age >= 2:

    print("Hello! Welcome.")

else:

    print("Good Bye!!")
```

Output

610

Hello! Welcome.

1.4.2 Operator Associativity in Python

If an expression contains two or more operators with the same precedence then Operator Associativity is used to determine. It can either be Left to Right or from Right to Left. The following code shows how Operator Associativity in Python works:

```
# Examples of Operator Associativity
# Left-right associativity
# 100 / 10 * 10 is calculated as
# (100 / 10) * 10 and not as 100 / (10 * 10)
print(100 / 10 * 10)
```

```
# Left-right associativity #

 5 - 2 + 3 is calculated as

# (5 - 2) + 3 and not as 5 - (2 + 3)

print(5 - 2 + 3)

# left-right associativity

print(5 - (2 + 3))

# right-left associativity

# 2 ** 3 ** 2 is calculated as

# 2 ** (3 ** 2) and not  as (2 ** 3) ** 2
```

Output

```
100.0
6
0
512
```

Lecture 3. Understanding python blocks & Data Types.

1.5 Understanding python blocks: A block in Python refers to a piece of code that performs a specific task. It can contain one or more statements and is defined by its indentation. Blocks are used to group statements together and provide structure to a program.

A block in Python is a group of one or more statements that perform a specific task. Blocks are defined by their indentation, which provides structure to the program. Indentation in Python is important as it defines the scope of a block and helps to keep the code organized.

For example, consider the following code:

```
if x > 0:
    print("x is positive")
    x = x + 1
```

In this code, the block is defined by the indentation of the two statements under the if clause. The block starts with the line print("x is positive") and ends with the line x = x + 1. The statements within the block will only be executed if the condition specified in the if clause is met.

Similarly, consider the following code:

```
for i in range(10):
    print(i)
```

In this code, the block is defined by the indentation of the statement underthe for clause. The block starts with the line print(i) and is executed once for each iteration of the loop.

It's important to note that blocks in Python can be nested, meaning that a block can contain one or more blocks within it. This allows for the creation of complex programs that can perform a variety of tasks.

Blocks in Python are essential to understand as they provide structure and organization to programs. By grouping related statements together, blocks make code easier to read and maintain.

1.6 Python Data Types: Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data. Since everything is an object in Python programming, data types are actually classes and variables are instances(object) of these classes.

The following is a list of the Python-defined data types.

1. Numbers
2. Sequence Type
3. Boolean

4. Set
5. Dictionary

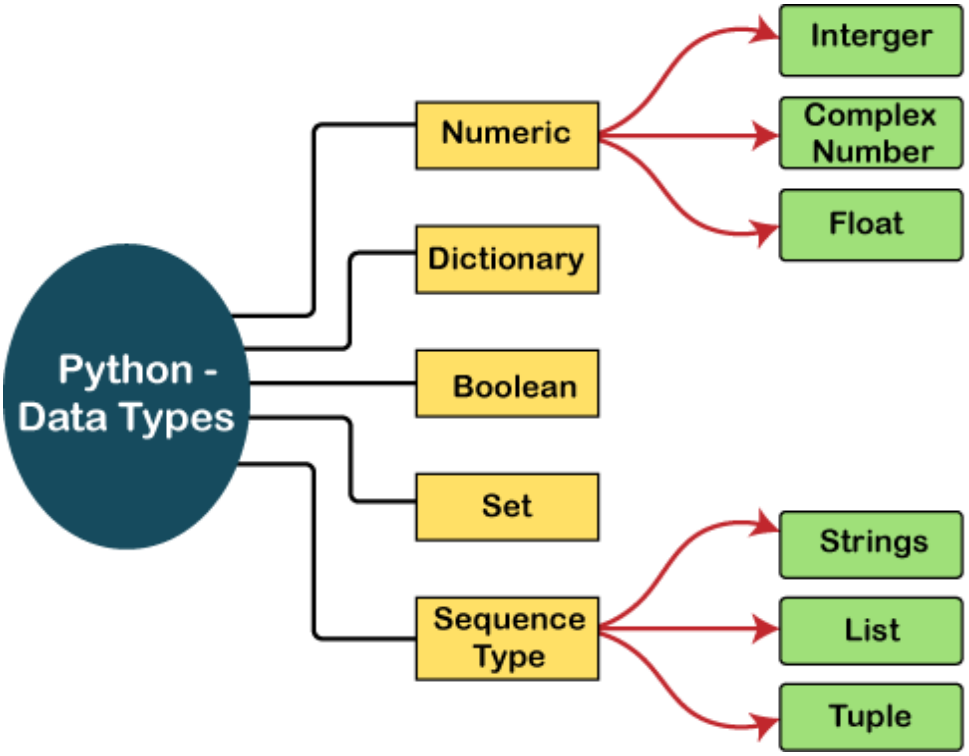


Figure 1 Python Data Types

Numbers

Numeric values are stored in numbers. The whole number, float, and complex qualities have a place with a Python Numbers datatype. Python offers the type() function to determine a variable's data type. The instance () capability is utilized to check whether an item has a place with a specific class.

When a number is assigned to a variable, Python generates Number objects. For instance,

```
1. a = 5
2. print("The type of a", type(a))3.
4. b = 40.5
5. print("The type of b", type(b))6.
7. c = 1+3j
8.  print("The type of c", type(c))
9.  print(" c is a complex number", isinstance(1+3j,complex))
```

Output:

```
The type of a <class 'int'> The
type of b <class 'float'>
The type of c <class 'complex'>c is
complex number: True
```

Lecture 4. Declaring and using Numeric data types.

1.6.1 Numeric Data Types-

Python supports three kinds of numerical data.

o Int: Whole number worth can be any length, like numbers 10, 2, 29, - 20, - 150, and so on. An integer can be any length you want in Python. Its worth has a place with int.

o Float: Float stores drifting point numbers like 1.9, 9.902, 15.2, etc. It can be accurate to within 15 decimal places.

o Complex: An intricate number contains an arranged pair, i.e., x + iy, where x and y signify the genuine and non-existent parts separately. The complex numbers like 2.14j, 2.0 + 2.3j, etc.

1.6.2 Sequence Type-

1.6.2.1 String

o The sequence of characters in the quotation marks can be used to describe the string. A string can be defined in Python using single, double, or triple quotes.

o String dealing with Python is a direct undertaking since Python gives worked-in capabilities and administrators to perform tasks in the string.

o When dealing with strings, the operation "hello"+" python" returns"hello python," and the operator + is used to combine two strings.

o Because the operation "Python" *2 returns "Python," the operator * is referred to as a repetition operator.

o The Python string is demonstrated in the following example.

Example - 1

```
1. str = "string using double quotes"
2. print(str)
3. s = """A multiline
4. string'"
5. print(s)
```

Output:

```
string using double quotesA
multiline
string
```

Look at the following illustration of string handling.

Example - 2

```
1. str1 = 'hello world' #string str1
2. str2 = ' how are you' #string str2
3. print (str1[0:2]) #printing first two character using slice operator
4. print (str1[4]) #printing 4th character of the string
```

```
5. print (str1*2) #printing the string twice
6. print (str1 + str2) #printing the concatenation of str1 and str2
```

Output:

```
he
o
hello worldhello world
hello world how are you
```

1.6.2.1 List

Lists in Python are like arrays in C, but lists can contain data of different types. The things put away in the rundown are isolated with a comma (,) and encased inside square sections [].

To gain access to the list's data, we can use slice [:] operators. Like how they worked with strings, the list is handled by the concatenation operator (+) and the repetition operator (*).

Look at the following example.

Example:

```
1. list1  = [1, "hi", "Python", 2]
2. #Checking type of given list
3. print(type(list
1))4.
5. #Printing the list1
6. print
(list1)7.
8. # List slicing
9. print
(list1[3:])10.
11.         # List slicing
12.         print
(list1[0:2])13.
14.         # List Concatenation using + operator
15.         print (list1 +
list1)16.
17.         # List repetation using * operator
18.         print (list1 * 3)
```

Output

```
[1, 'hi', 'Python', 2]
[2]
[1, 'hi']
[1, 'hi', 'Python', 2, 1, 'hi', 'Python', 2]
```

[1, 'hi', 'Python', 2, 1, 'hi', 'Python', 2, 1, 'hi', 'Python', 2]

1.6.2.2 Tuple

In many ways, a tuple is like a list. Tuples, like lists, also contain a collection of items from various data types. A parenthetical space () separates the tuple'scomponents from one another.

Because we cannot alter the size or value of the items in a tuple, it is a read- only data structure.

Let's look at a straightforward tuple in action.

Example:

```
  tup  = ("hi", "Python", 2)
 # Checking type of tup
 print (type(tup))
 4.
 #Printing the tuple
 print (tup)
 7.
 # Tuple slicing
 print (tup[1:])
 print (tup[0:1])11.
 # Tuple concatenation using + operator
  print (tup + tup)14.
 # Tuple repatation using * operator
 print (tup * 3)17.
 # Adding value to tup. It will throw an error.19.
 t[2] = "hi"
```

Output-

```
<class 'tuple'> ('hi',
'Python', 2)
('Python', 2)
('hi',)
('hi', 'Python', 2, 'hi', 'Python', 2)
('hi', 'Python', 2, 'hi', 'Python', 2, 'hi', 'Python', 2)

Traceback (most recent call last):
  File "main.py", line 14, in <module>t[2] =
    "hi";
TypeError: 'tuple' object does not support item assignment
```

1.6.3 Dictionary

A dictionary is a key-value pair set arranged in any order. It stores a specific value for each key, like an associative array or a hash table. Value is any Python object, while the key can hold any primitive data type.

The comma (,) and the curly braces are used to separate the items in the dictionary.

Look at the following example.

```
d = {1:'Jimmy', 2:'Alex', 3:'john', 4:'mike'}
# Printing dictionary
print (d)
# Accesing value using keys
print("1st name is "+d[1])
print("2nd name is "+ d[4]).
print (d.keys())
print (d.values())
```

Output

```
1st name is Jimmy2nd name is
mike
{1: 'Jimmy', 2: 'Alex', 3: 'john', 4: 'mike'}
dict_keys([1, 2, 3, 4])
dict_values(['Jimmy', 'Alex', 'john', 'mike'])
```

1.6.4 Boolean

True and False are the two default values for the Boolean type. These qualities are utilized to decide the given assertion valid or misleading. The class book indicates this. False can be represented by the 0 or the letter "F," while true can be represented by any value that is not zero.

Look at the following example.

```
1. # Python program to check the boolean type
2. print(type(True))
3. print(type(False))
4. print(false)
```

Output:

```
<class 'bool'>
<class 'bool'>
NameError: name 'false' is not defined
```

1.6.5 Set

The data type's unordered collection is Python Set. It is iterable, mutable(can change after creation), and has remarkable components. The elements of a sethave no set order; It might return the element's altered sequence. Either a sequence of elements is passed through the curly braces and separated by a comma to create the set or the built-in function set() is used to create the set. It can contain different kinds of values.

Look at the following example.

```
# Creating Empty set
set1 = set()3.
```

```
set2 = {'James', 2, 3,'Python'}
#Printing Set value
print(set2)
# Adding element to the set
set2
add
print(set2)
#Removing element from the set
set2
remove(2)
print(set2)
```

Output:

```
{3, 'Python', 'James', 2}
{'Python', 'James', 3, 2, 10}
{'Python', 'James', 3, 10}
```

Important Questions Unit-1

i. Discuss why python is an interpreted language ? Explain history and features of python.(2021-22)

ii. Explain type conversion in python with an example.(2021-22)

iii. Describe Arithmetic Operators, Assignment Operators, Comparison Operators, Logical Operators and Bitwise Operators in detail with examples. (2020-21).

iv. Discuss why Python is called a dynamic and strongly-typed language.

v. Which of the following statements will produce an error in python?(2020-21)

S1 x,y,z= 1,2,3

S2 a,b=4,5,6

S3 u = 7,8,9

(list all the statements that have error)

vi. Explain the Programming Cycle for Python in detail.(2021-22,2022-23)

UNIT-2

Lecture 5. Flow Control Conditional if, else and else if

In Python, flow control structures such as if, else, and elif (else if) are used to implement conditional logic. These constructs allow you to control the flow of your program based on different conditions.

Let's discuss how to use them:

2.1 Control flow statements in Python

First and foremost, Control flow statements in Python are how you direct your programs to decide which parts of code to run. By default, programs execute each line of code in sequence, with understanding how things are proceeding.

What if you don't want to execute every single line of code? What if you have several answers, and the code must choose which one to utilize based on the conditions? What if you require the software to continually utilize the same code to perform the computations with slightly different inputs? What if you want to execute a few lines of code repeatedly until the application fulfills a condition? It is when control flow enters the picture. Control flow statements in Python direct the flow of your program's execution. It allows you to make decisions, repeat actions, and handle different situations to make your code more dynamic and adaptable.

2.2 Conditional Statements in Python

Conditional statements in Python are used to make decisions and execute different blocks of code based on specific conditions. These conditions are defined using logical expressions that evaluate either True or False. Conditional statements in Python control the flow of your program and enable it to respond dynamically to different situations.

2.2.1 if Statement:

The if statement is the most fundamental conditional statement in Python. It allows you to execute a block of code only if a specified condition is true. If the condition evaluates to True, the code block is executed.

The if statement is used to execute a block of code only if a certain condition is true.

```
# Example of the if statement

x = 10
if x > 5:
    print("x is greater than 5")
Output: x is greater than 5
```

2.2.2 if else Statement: The else statement is used to execute a block of code if the condition specified in the if statement is false.

```
# Example of the if-else statement
y = 3
if y > 5:
  print("y is greater than 5")

else:
  print("y is not greater than 5")
```

Output: y is not greater than 5

2.2.3 elif Statement:

The elif (else if) statement is used when there are multiple conditions to check. It allows you to check additional conditions if the previous ones are false.

```
# Example of the if-elif-else statement

z = 0
if z > 0:
print("z is positive")

elif z < 0:
  print("z is negative")
else:
  print("z is zero")
```

Output: z is zero

Lecture 6. Simple for loops, for loop using ranges.

2.3 Loops in Python

Iterative control statements in Python allow you to execute a block of code repeatedly. They perform repetitive tasks, iterate over data structures, and handle various scenarios where actions need to be repeated.

Python has two main types of loops:

- The for loop
- The while loop

2.3.1 The for Loop

The for loop iterates over a sequence (such as a list, tuple, string, or range) or any other iterable object. It executes ablock of code for each element in the sequence to perform actions on each element.

Example 1: Using for Loop with a List

Code

```
fruits = ["apple," "banana," "cherry"]

for fruit in fruits:

  print(f"I love {fruit}s.")
```

Output:

code

I love apples.

I love bananas.

I love cherries.

In this example, the for loop iterates through the fruits list, and for each fruit, it executes the code block inside

theloop. This results in the message "I love [fruit]s." being printed for each item in the list.

Example 2: Using for Loop with a String

code

```
word = "Python"

for  letters in words:

    print(letter)
```

Output:

P
y
t
h
o
n

Here, the for loop iterates through the characters of the string "Python" and prints each character on a separate line.

Example 3: Using for Loop with Range

code

```
for i in range(1, 6):
  print(f" The Square of {i} is {i**2}.")
```

Output:

The square of 1 is 1.

The square of 2 is 4.

The square of 3 is 9.

The square of 4 is 16.

The square of 5 is 25.

This example uses the range() function to generate a sequence of numbers from 1 to 5 (inclusive). The for loop theniterates through this sequence, calculating and printing the square of each number.

2.3.2 The while Loop

The while loop in Python repeatedly executes a block of code as long as a specified condition remains true. It isuseful when you need to repeat an action until a certain condition is met.

Example 1: Basic

while LoopCode

```
count = 1

while count <= 5:

    print(f"The Count is {count}.")

    count+ = 1
```

Output:

code
The count is 1.

The count is 2.

The count is 3.

The count is 4.

The count is 5.

In this example, the while loop continues to run as long as the condition (count <= 5) remains true.

Example 2: Using while Loop for User Input

Code

```
password = "secret"

user_input = input("Enter the password: )

while user_input != password:

    print("Incorrect password. Try again.")

    user_input = input("Enter the password: ")

print("Access granted.")
```

Output (assuming incorrect password inputs before entering "secret"):

```
Enter the password: incorrect Incorrect password. Try
again.
Enter the password: wrong pass
Incorrect password. Try again.
Enter the password: secret
Access granted.
```

This example uses a while loop to repeatedly prompt the user for a password until they enter the correct password("secret").

Example 3: Using While Loop with a Counter

Code

```
counter = 0
while counter < 3:
        print(f"Processing item {counter}”)
       counter+ = 1
```

Output:

```
Processing item 1

Processing item 2

Processing item 3
```

Here, the while loop is used to process items in a task. The loop continues until the counter reaches 3, at whichpoint it stops.

Lecture 7. String

2.4. String- Python string is the collection of the characters surrounded by single quotes, double quotes, or triple quotes. The computer does not understand the characters; internally, it stores manipulated character as the combination of the 0'sand 1's.

Each character is encoded in the ASCII or Unicode character

a string is a sequence of characters enclosed in single (' '), double (" "), or triple ('" "' or """ """) quotes. Strings areimmutable, meaning their values cannot be changed after creation.

some key aspects of working with strings in Python:

2.4.1 Creating Strings:

```
# Single quotes
single_quoted = 'Hello, Python!'


# Double quotes
double_quoted = "String in Python."


# Triple quotes for multiline string
smultiline = '"This is a
multiline string."'
```

2.4.2 Accessing Characters in a String:

```
my_string = "Python"
# Accessing individual characters
first_char = my_string[0] #’P’

last_char = my_string[-1]  # 'n'
```

2.4.3 String Slicing:

```
# Slicing a portion of a string
substring = my_string[1:4]  #
'yth'
```

2.4.4 String Concatenation:

```
string1 = "Hello"
string2 = "Python"
```

```
# Concatenating strings
concatenated = string1 + " " + string2  # 'Hello Python'
```

2.4.5 String Methods:

Python provides various built-in string methods for manipulating and working with

```
strings.my_string = " Hello, Python! "

# Removing leading and trailing whitespaces
trimmed_string = my_string.strip()  # 'Hello,
Python!'


# Converting to lowercase
lowercased = my_string.lower()  # ' hello, python! '


# Converting to uppercase
uppercased = my_string.upper()  # ' HELLO, PYTHON! '


# Finding substring
index = my_string.find("Python")  # 8
```

2.4.6 String Formatting:

```
name = "Alice"
age = 30


# Using f-strings (Python 3.6 and above)
formatted_string = f"My name is {name} and I am {age} years old."


# Using the format method
formatted_string2 = "My name is {} and I am {} years old.".format(name, age)
```

2.4.7 String Operations:

```
# String length
length = len(my_string)  #15
# Check if a substring is present
```

```
contains_python = "Python" in my_string  #
True
# Repeat a string
repeated_string = my_string * 3  # ' Hello, Python! Hello, Python! Hello, Python! '
```

2.4.8 Escape Characters:

```
escaped_string = "This is a line.\nThis is a new line."
# Output:
This is a line.
This is a new line.
```

Lecture 8- List and dictionaries

2.5 Lists: A list is a versatile and mutable data structure in Python that can hold an ordered collection of items. Lists are defined using square brackets **[ ]** and can contain elements of different data types.

2.5.1 Creating Lists:# Empty list

```
empty_list = []
```

2.5.2 List with elements

```
fruits = ['apple', 'banana', 'orange']
```

2.5.3 List with mixed data types

```
mixed_list = [1, 'two', 3.0, True]
```

2.5.4 Accessing List Elements:

```
# Accessing individual elements
first_fruit = fruits[0]  # 'apple'
second_fruit = fruits[1]  # 'banana'
```

2.5.5 Slicing a portion of a list

```
subset = fruits[1:3]
# ['banana', 'orange']
```

2.5.6 Modifying Lists:

```
# Modifying an element
fruits[0] = 'kiwi'
```

2.5.7 Appending to the end of the list

```
fruits.append('grape')
```

2.5.8 Extending with another list

```
fruits.extend(['pineapple', 'watermelon'])
```

2.5.9 List Operations:

 # Length of a list

```
length = len(fruits)  # 5
```

Check if an item is in the list

```
contains_banana = 'banana' in fruits  # True
```

Concatenating list

```
more_fruits = ['pear', 'plum']
combined_list = fruits + more_fruits
```

2.6 Dictionaries:

A dictionary is an unordered collection of key-value pairs. It is defined using curly braces { } and colons : to separatekeys and values. Dictionaries are commonly used for data that needs to be looked up quickly.

2.6.1 Creating Dictionaries:

 # Empty dictionary

```
empty_dict = {}
```

 # Dictionary with key-value pairs

```
student = {'name': 'Alice', 'age': 25, 'grade': 'A'}
```

2.6.2 Accessing Dictionary Elements:

 # Accessing values by key

```
student_name = student['name']  # 'Alice'
```

 # Using the get method to avoid KeyErrors

```
student_age = student.get('age')  # 25
```

2.6.3 Modifying and Adding to Dictionaries:

 # Modifying a value

```
student['age'] = 26
```

 # Adding a new key-value pair

student['city'] = 'Wonderland'

2.6.4 Dictionary Operations:

 # Length of a dictionary (number of key-value pairs)

dict_length = len(student) # 4

 # Checking if a key is in the dictionary

has_grade = 'grade' in student # True

 # Removing a key-value pair removed_grade =

student.pop('grade')

Lecture 9. Use of while loops in python

2.7 While loop- In Python, a while loop is used to repeatedly execute a block of code as long as a specified condition is true. The loopcontinues to execute until the condition becomes false. Let's discuss how to use while loops in Python:

2.7.1 Basic while Loop:

```
# Example of a basic while loop
count = 0
while count < 5:
    print(f"Count: {count}")
    count += 1
```

2.7.2 Infinite Loop:

```
# Example of an infinite while loop
while True:
    user_input = input("Enter a number (type 'exit' to stop): ")
    if user_input.lower() == 'exit':
        break          # Exit the loop if 'exit' is entered
    else:
        print(f"You entered: {user_input}")
```

2.7.3 Using else with while:

```
# Example of using else with while
loopnum = 0

while num < 5:

  print(f"Num: {num}")
```

```
 num += 1
else:
print("Loop finished.")
```

2.7.4 while Loop with continue:

```
# Example of using continue in a while loopnum = 0
while num < 5:
  num += 1
   if num == 3:
       continue         # Skip the rest of the code
in the loop for num == 3

print(f"Num: {num}")
```

The continue statement is used to skip the remaining code inside the loop for the current iteration and move to thenext iteration.

2.7.5 while Loop with else and break:

```
# Example of using else and break in a while loopnum = 0
while num < 5:'
    print(f"Num: {num}")
    num += 1
    if num == 3:
      break  # Exit the loop when num == 3else:
print("Loop finished.")
```

In this example, the else block is not executed because the loop is terminated by the break statement.

Lecture 10. Loop manipulation using pass continue, break and else.

2.8 Manipulation Statements- In Python, loop manipulation statements like pass, continue, break, and else provide ways to control the flow of loops.

2.8.1 pass Statement:

The pass statement is a no-operation statement that serves as a placeholder when syntactically some code is required,but you don't want to execute any specific operation.

```
for i in range(5):
    if i == 2:
        pass # Do nothing when i is 2
    else:
        print(i)
# Output:
# 0
# 1
# 3
# 4
```

2.8.2 continue Statement:

The continue statement is used to skip the rest of the code inside the loop for the current iteration and move to thenext iteration.

```
for i in range(5):
 if i == 2:
     continue  # Skip printing when i is 2
   print(i)
#Output:
# 0
# 1
# 3
```

4

2.8.3 break Statement:

The break statement is used to exit the loop prematurely. It terminates the loop when a specified condition is met.

```
for i in range(5):
    if i == 3:
        break  # Exit the loop when i is 3
    print(i)
# Output:
# 0
# 1
# 2
```

2.9 else Clause in Loops:

The else clause in a loop is executed when the loop condition becomes False. However, if the loop is terminated by abreak statement, the else clause is skipped.

```
for i in range(5):
 print(i)
else:
print("Loop finished.")
# Output:
# 0
# 1
# 2
# 3
# 4
# Loop finished.
for i in range(5):
   if i == 3:
      break  # Exit the loop when i is 3print(i)
else:
 print("Loop finished.")
# Output:
# 0
#1
#2
```

Lecture 11. Condition and loop blocks in Program.

2.10Condition Blocks: Condition blocks are used to execute specific blocks of code based on whether a certain condition is true or false.

2.10.1 if Statement:

```
x = 10
if x > 5:
  print("x is greater than 5")
```

2.10.2 else Statement:

```
y = 3
if y > 5:
   print("y is greater than 5")
else:
   print("y is not greater than 5")
```

2.10.3 elif Statement:

```
z = 0
if z > 0:
  print("z is positive")
elif z < 0:
  print("z is negative")
else:
  print("z is zero")
```

2.11 Loop Blocks:

Loop blocks are used for repeating a block of code multiple times.

for Loop:

```
fruits = ['apple', 'banana', 'orange']
for fruit in fruits:
    print(fruit)
```

while Loop:

```
count = 0
while count < 5:
  print(f"Count: {count}")
  count += 1
```

2.12 Combining Condition and Loop Blocks:

You can also use conditionals within loops to create more complex control structures.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]

for num in numbers:
 if num % 2 == 0:
    print(f"{num} is even")
 else:
    print(f"{num} is odd")
```

2.13 Nested Blocks:

You can nest conditionals and loops within each other for more intricate program logic.

```
for  i in range(3):
if i == 0:
    print("First iteration")
 else:
    print("Not the first iteration")
```

Important Ques Unit-2

i. How pass statement is different from comment? (2021-22)
ii. Explain the use of BREAK and CONTINUE statement with example . (2022-23)
iii. In Some languages, every statement ends with a semicolon(;) , what happens if you put a semi-colon at the end of a python statement.(2019-20)
iv. Explain the purpose and working of loops with their flowchart , syntax and suitable example.(2022-23)
v. Write a python program to construct the following pattern , using a nested for loop.(2021-22)

```
        *
```

```
        * *
        * * *
        * * * *
        * * * * *
        * * * *
        * * *
        * *
        *
```

UNIT 3

Lecture 12. Use of string data type & string operations.

3.1 String

- A String is a data structure in Python that represents a sequence of characters.
- It is an immutable data type, meaning that once you have created a string, you cannot change it.
- Strings are used widely in many different applications, such as storing and manipulating text data, representing names, addresses, and other types of data that can be represented as text.

Python string is the collection of the characters surrounded by single quotes, double quotes, or triple quotes. Python does not have a character data type, a single character is simply a string with a length of 1.

3.1.1 Creating String in Python

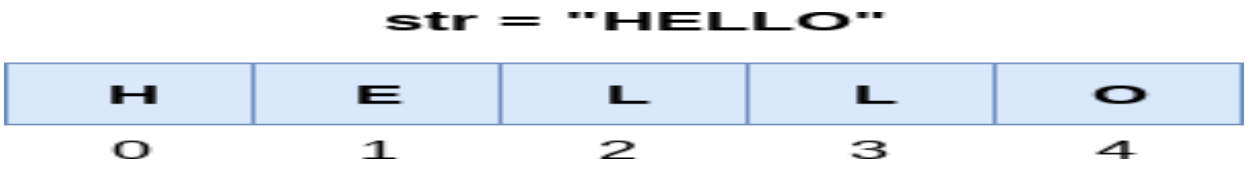
We can create a string by enclosing the characters in single-quotes or double- quotes. Python also provides triple-quotes to represent the string, but it is generally used for multiline string or **docstrings**.

```
# Creating a String
# with single Quotes
String1 = 'Welcome to the Geeks World'
print("String with the use of Single Quotes: ")
print(String1)

# Creating a String
# with double Quotes
String1 = "I'm a Geek"
print("\nString with the use of Double Quotes: ")
print(String1)

# Creating a String
# with triple Quotes
String1 = '''I'm a Geek and I live in a world of "Geeks"'''
print("\nString with the use of Triple Quotes: ")
print(String1)

# Creating String with triple
# Quotes allows multiple lines
String1 = '''Geeks
        For
        Life'''
print("\nCreating a multiline String: ")
print(String1)
```

```
String with the use of Single Quotes:
Welcome to the Geeks World
String with the use of Double Quotes:
I'm a Geek
String with the use of Triple Quotes:
I'm a Geek and I live in a world of "Geeks"
Creating a multiline String:
Geeks
        For
        Life
```

3.1.3 Accessing characters in Python String

- In Python, individual characters of a String can be accessed by using the methodof Indexing.
- Indexing allows negative address references to access characters from the back ofthe String, e.g. -1 refers to the last character, -2 refers to the second last character,and so on.
- While accessing an index out of the range will cause an **IndexError**. Only Integersare allowed to be passed as an index, float or other types that will cause a **TypeError**.



3.1.4.String Slicing

- In Python, the String Slicing method is used to access a range of characters in the String. Slicing in a String is done by using a Slicing operator, i.e., a colon (:).
- One thing to keep in mind while using this method is that the string returned after slicing includes the character at the start index but not the character at the last index.

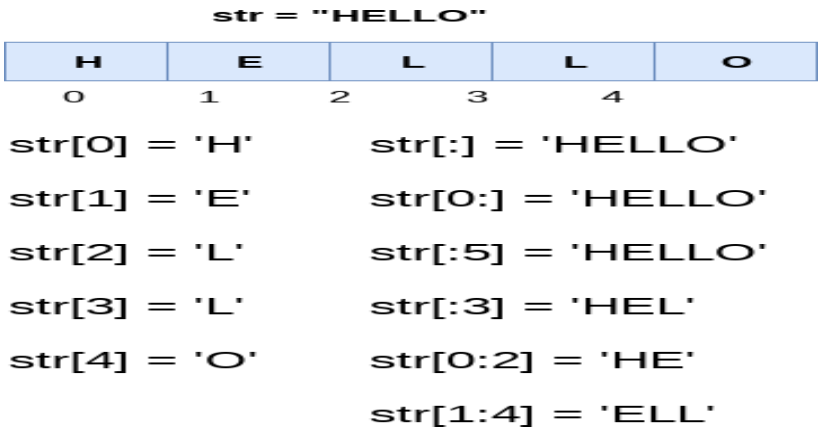


Figure 3.1 String Slicing

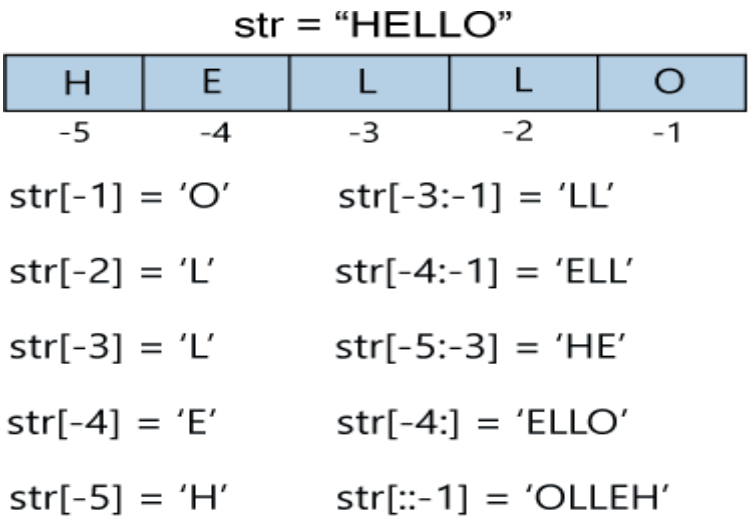


Figure 3.2 String Slicing

3.1.5 Reassigning Strings

As Python strings are immutable in nature, we cannot update theexisting string. We can only assign a completely new value to the variable with the samename.

Consider the following example.

Example 1

```
str = "HELLO"
str[0] = "h"
print(str)
```

```
Traceback (most recent call last):
  File "12.py", line 2, in <module>
    str[0] = "h";
TypeError: 'str' object does not support item assignment
```

Output:

A character of a string can be updated in Python by first converting the string into a Python List and then updating the element in the list. As lists are mutable in nature, we can update the character and then convert the list back into the String.

```
# Python Program to Update
# character of a String

String1 = "Hello, I'm a Geek"
print("Initial String: ")
print(String1)
```

```
# Updating a character of the String
## As python strings are immutable, they don't support item updation directly
### there are following two ways
#way1
list1 = list(String1)
list1[2] = 'p'
String2 = ''.join(list1)
print("\nUpdating character at 2nd Index: ")
print(String2)

#way 2
String3 = String1[0:2] + 'p' + String1[3:]
print(String3)
```

Output:

```
Initial String:
Hello, I'm a Geek
Updating character at 2nd Index:
Heplo, I'm a Geek
Heplo, I'm a Geek
```

3.1.6 Deleting the String

As we know that strings are immutable. We cannot delete or remove the characters from the string. But we can delete the entire string using the **del** keyword.

```
str = "JAVATPOINT"
del str[1]
```

Output:

```
TypeError: 'str' object doesn't support item deletion
```

Now we are deleting entire string.

```
str1 = "JAVATPOINT"
del str1
print(str1)
```

Output:

```
NameError: name 'str1' is not defined
```

3.1.7 Escape Sequencing in Python

- While printing Strings with single and double quotes in it causes **SyntaxError** because String already contains Single and Double Quotes and hence cannot be printed with the use of either of these.
- Escape sequences start with a backslash and can be interpreted differently. If single quotes are used to represent a string, then all the single quotes present in the string must be escaped and the same is done for Double Quotes.

Example

```
String1= "C:\\Python\\Geeks\\"
print(String1)
```

```
C:\Python\Geeks\
```

3.1.8 Formatting of Strings

Strings in Python can be formatted with the use of format( ) . Format method in String contains curly braces {} as placeholders which can hold arguments according to position or keyword to specify the order.

```
String1 = "{l} {f} {g}".format(g='Geeks', f='For', l='Life')
print("\nPrintString in order of Keywords: ")
print(String1)
```

Output

```
Print String in order of Keywords:
Life For Geeks
```

Letcure 13. Python List & List Slicing

3.2 List-

- The list is a sequence data type which is used to store the collectionof data.
- In lists the comma (,)and the square brackets [enclose the List's items] serveas separators.
- Lists need not be homogeneous always which makes it the most powerfultool in Python.
- A single list may contain DataTypes like Integers, Strings, as well as Objects.
- Lists are mutable, and hence, they can be altered even after their creation.

3.2.1 Creating a List in Python

Lists in Python can be created by just placing the sequence inside the square brackets[].

A list may contain duplicate values with their distinct positions and hence, multiple distinctor duplicate values can be passed as a sequence at the time of list creation.

Example-

List1=[“physics”, “Maths”,34,15.5]

List2=[2,3,5,10]

3.2.2 List Indexing and Splitting

The indexing procedure is carried out similarly to string processing. The slice operator [] canbe used to get to the List's components.

The index ranges from 0 to length -1.

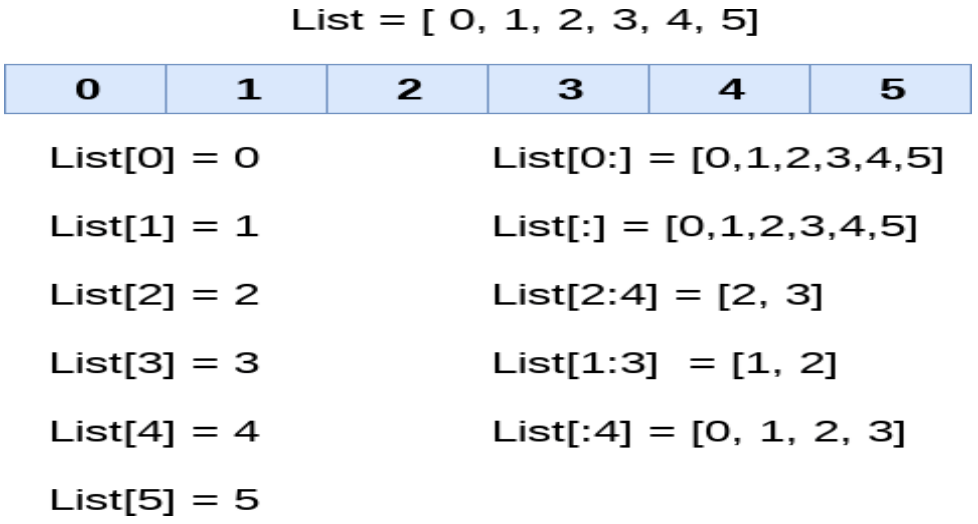
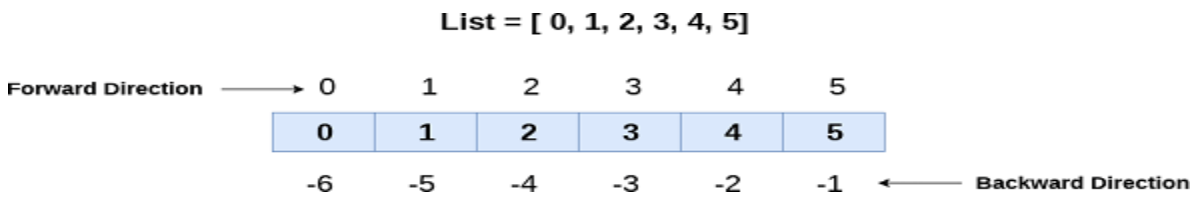


Figure 3.3 List Slicing



We can get the sub-list of the list using the following syntax.

```
1. list_varible(start:stop:step)
```

- The beginning indicates the beginning record position of the rundown.
- The stop signifies the last record position of the rundown.
- Within a start, the step is used to skip the nth element: stop.

Example-
List1=[0,1,2,3,4]

List1[1:3:1]= [1,2]

3.2.3 Adding Elements to a Python List

Method 1: Using append() method -

Only one element at a time can be added to the list by using the append() method

```
List = []
print("Initial blank List: ")
print(List)

List.append(1)
List.append(2)
print("List after Addition of Three elements: ")
print(List)
```

```
Initial blank List:
[]

List after Addition of
Three elements:
[1, 2]
```

Method 2: Using insert() method-
append() method only works for the addition of elements at the end of the List, forthe addition of elements at the desired position insert() method is used.

```
List.insert(2, 12)
print("List after performing Insert Operation: ")
print(List)
```

```
List after performing
Insert Operation:
[1,2,12]
```

Method 3: Using extend() method-

Extend( ) method is used to add multiple elements at the same time at the end ofthe list.

```
List.extend([8, 'Geeks'])
print("List after performing Extend Operation: ")
print(List)
```

```
List after performing Extend
Operation:
[1,2,12,8,Geeks]
```

3.2.4 Removing Elements from the List

Method 1: Using remove() method

Elements can be removed from the List by using the built-in remove() function but an Error arises if the element doesn't exist in the list.

```
List = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
List.remove(5)
List.remove(6)
print("List after Removal of two elements: ")
print(List)
```

```
List after Removal of two
elements:
[1,2,3,4,7,8,9,10,11,12]
```

Method 2: Using pop() method

pop() function can also be used to remove and return an element from the list, but by default it removes only the last element of the list, to remove an element from a specific position of the List, the index of the element is passed as an argument to the pop() method.

```
List = [1, 2, 3, 4, 5]
List.pop()
print("List after popping an element: ")
print(List)
List.pop(2)
print("\nList after popping a specific element: ")
print(List)
```

```
List after popping an element:
[1,2,3,4]

List after popping a specific
element:
[1,2,4]
```

3.3 List Operations-

Basic List Operations

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

In fact, lists respond to all of the general sequence operations we used on strings in the prior chapter.

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1, 2, 3]: print x,	1 2 3	Iteration

Table 3.1 List Operations

3.4 List out some in-built function in python List

SN	Function	Description
1	cmp(list1, list2)	It compares the elements of both the lists.
2	len(list)	It is used to calculate the length of the list.
3	max(list)	It returns the maximum element of the list.
4	min(list)	It returns the minimum element of the list.
5	list(seq)	It converts any sequence to the list.

Table 3.2 List In Built Function

3.5 List Built In Methods-

SN	Function	Description
1	list.append(obj)	The element represented by the object obj is added to the list.
2	list.clear()	It removes all the elements from the list.
3	List.copy()	It returns a shallow copy of the list.
4	list.count(obj)	It returns the number of occurrences of the specified object in the list.
5	list.extend(seq)	The sequence represented by the object seq is extended to the list.
6	list.index(obj)	It returns the lowest index in the list that object appears.
7	list.insert(index, obj)	The object is inserted into the list at the specified index.
8	list.pop(obj=list[-1])	It removes and returns the last object of the list.
9	list.remove(obj)	It removes the specified object from the list.
10	list.reverse()	It reverses the list.
11	list.sort([func])	It sorts the list by using the specified compare function if given.

Table 3.3 List InBuilt Methods

Ques : Python Program to find the second largest number in a list.

Ans:

```
a=[]
n=int(input("Enter number of elements:"))
 for i in range(1,n+1):
b=int(input("Enter element:"))
    a.append(b)
a.sort()
print("Second largest element is:",a[n-2])
```

Ques: Program to remove the duplicate items from a list.
Ans:

```
a=[]
n= int(input("Enter the number of elements in list:"))
for x in range(0,n):
element=int(input("Enter element" ))
    a.append(element)
b = []
[b.append(x) for x in a if x not in b]
print("Non-duplicate items:")
print(unique)
```

Lecture 14. How to Use of Tuple data type

3.6 Tuple

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally you can put these comma-separated values between parentheses also. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5 );
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing −

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value −

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

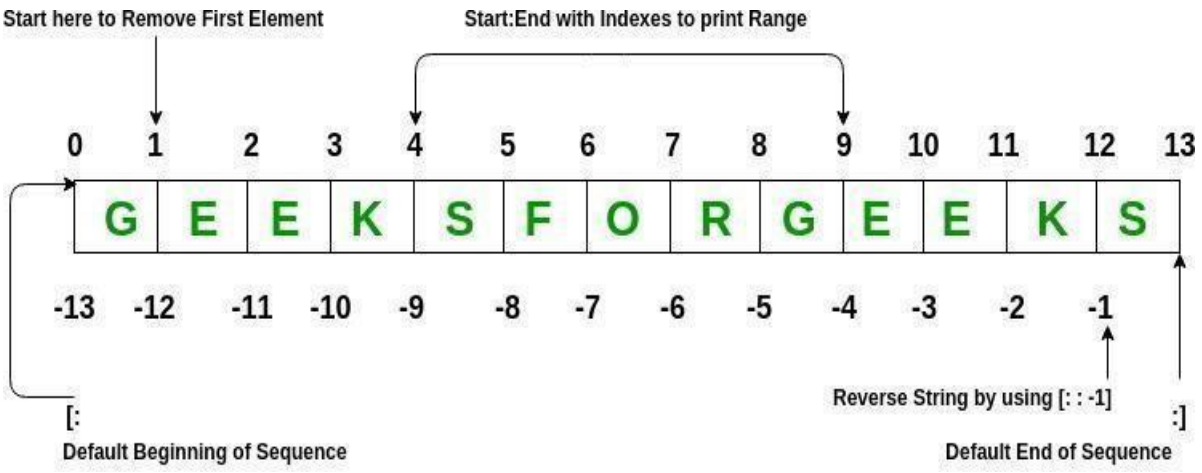
Note: Creation of Python tuple without the use of parentheses is known as TuplePacking.

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example −

```
tup1 = ('physics', 'chemistry', 1997, 2000);
tup2 = (1, 2, 3, 4, 5, 6, 7 );
print "tup1[0]: ", tup1[0];
print "tup2[1:5]: ", tup2[1:5];
```

When the above code is executed, it produces the following result −

```
tup1[0]:  physics
tup2[1:5]:  [2, 3, 4, 5]
```



Note: We can change any python collection like (list, tuple, set) and string into other data type like tuple, list and string. But cannot change into and float to any python collection. For example: int cannot change into list or vice-versa.

Deleting a Tuple

Tuples are immutable and hence they do not allow deletion of a part of it. The entire tuple gets deleted by the use of del() method.

Note- Printing of Tuple after deletion results in an Error.

Python

```
Tuple1 = (0, 1, 2, 3, 4)

del Tuple1

print(Tuple1)
```

Traceback (most recent call last):

File "/home/efa50fd0709dec08434191f32275928a.py", line 7, in

print(Tuple1)

NameError: name 'Tuple1' is not defined

3.6.1 Tuple Operations

Basic Tuples Operations

Tuples respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new tuple, not a string.

In fact, tuples respond to all of the general sequence operations we used on strings in the prior chapter −

Python Expression	Results	Description
len((1, 2, 3))	3	Length
(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)	Concatenation
('Hi!',) * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	Repetition
3 in (1, 2, 3)	True	Membership
for x in (1, 2, 3): print x,	1 2 3	Iteration

Table 3.4 Tuple Operations

3.6.2 Tuple Built-in Functions

Built-in Tuple Functions

Python includes the following tuple functions −

Sr.No.	Function with Description
1	cmp(tuple1, tuple2) Compares elements of both tuples.
2	len(tuple) Gives the total length of the tuple.
3	max(tuple) Returns item from the tuple with max value.
4	min(tuple) Returns item from the tuple with min value.
5	tuple(seq) Converts a list into tuple.

Table 3.5 Tuple Built-in Functions

3.7 The zip() Function

The zip() is an inbuilt function in python. It takes items in sequence from a number of collections to make a list of tuples, where each tuple contains one item from each collection. The function is often used to group items from a list which has the same index.

Example:

```
A=[1,2,3]

B=’XYZ’

Res=list(zip(A,B))  # list of tuples
print(Res)
```

Output:

```
[(1, 'X'), (2, 'Y'), (3, 'Z')]
```

We can change into tuple of tuples

```
Res=tuple(zip(A,B))

# tuple of tuples
```

Note: if the sequences are not of the same length then the result of zip() has the length of the shorter sequence.

```
A=’abcd

B=[1,2,3]

Res=list(zip(A,B))

print(Res)
```

Output:

```
[(‘a’,1),(‘b’,2),(‘c’,3)]
```

The Inverse zip(*) Function

The * operator is used within the zip() function. The * operator unpacks a sequence into positional arguments.

```
X=[(‘apple’,90000),(‘del’,60000),(‘hp’,50000)]

Laptop,prize=zip(*X)

print(Laptop)

print(prize)
```

Output:

```
(‘apple’,’del’,’hp’)

(90000,60000,50000)
```

Python program to find the maximum and minimum K elements in a tuple

```
# Python program to find maximum and minimum k elements in tuple

# Creating a tuple in python
myTuple = (4, 9, 1, 7, 3, 6, 5, 2)

K = 2


# Finding maximum and minimum k elements in tuple
sortedColl = sorted(list(myTuple))

vals = []

for i in range(K):
    vals.append(sortedColl[i])

for i in range((len(sortedColl) - K), len(sortedColl)):
    vals.append(sortedColl[i])

# Printing

print("Tuple : ", str(myTuple))

print("K maximum and minimum values : ", str(vals))
```

Output

```
Tuple : (4, 9, 1, 7, 3, 6, 5, 2)
```

Lecture 15. String and string manipulation methods.

3.8 string manipulation methods-

capitalize()	**Converts the first character to upper case**
count()	**Returns the number of times a specified value occurs in a string**
endswith()	**Returns true if the string ends with the specified value**
find()	**Searches the string for a specified value and returns the position of where itwas found**
format()	**Formats specified values in a string**
index()	**Searches the string for a specified value and returns the position of where itwas found**
isalnum()	**Returns True if all characters in the string are alphanumeric**
isalpha()	**Returns True if all characters in the string are in the alphabet**
isascii()	**Returns True if all characters in the string are ascii characters**
isdecimal()	**Returns True if all characters in the string are decimals**
isdigit()	**Returns True if all characters in the string are digits**
islower()	**Returns True if all characters in the string are lower case**
isspace()	**Returns True if all characters in the string are whitespaces**
istitle()	**Returns True if the string follows the rules of a title**

isupper()	**Returns True if all characters in the string are upper case**
join()	**Joins the elements of an iterable to the end of the string**
split()	**Splits the string at the specified separator, and returns a list**
replace()	**Returns a string where a specified value is replaced with a specified value**
startswith()	**Returns true if the string starts with the specified value**
title()	**Converts the first character of each word to upper case**
upper()	**Converts a string into upper case**
lower()	**Converts a string into lower case**
swapcase()	**Swaps cases, lower case becomes upper case and vice versa**

Table 3.6 ***string manipulation methods***

Programs on Python Strings

1. WAP to display unique words from the string

```
str=input("Enter string:") l=str.split()
l1=[]
for i in l:
  if i not in l1:
    l1.append(i)
str=" ".join(l1)
print(str)
```

2.WAP to accept a string & replace all spaces by # without using string method.

```
str=input("Enter string:") str1=""
for i in str:
 if i.isspace():
    str1+="#
 else:
        str1+=i
print("output is",str1)
```

Lecture 16 Python Set and list manipulating methods

3.9 Set- set is an unordered collection of data items which does not contain duplicate values. Every set element is unique and must be immutable (cannot be changed). However, a set itself is mutable. Elements of set are enclosed inside a pair of curly brackets {}.

We can add and remove items from the set. But cannot replace item as it does not support indexing of data items.

Empty set is created by:

A=set()

print(A)

Output:

Set()

Access Items

You cannot access items in a set by referring to an index, since sets are unordered the items has no index.

But you can loop through the set items using a `for` loop, or ask if a specified value is present in a set, by using the `in` keyword.

Example

Loop through the set, and print the values:

```
thisset = {"apple", "banana", "cherry"}

for x in thisset:
  print(x)
```

Output**:**
apple
banana
cherry

Change Items

Once a set is created, you cannot change its items, but you can add new items.

Add Items

To add one item to a set use the `add()` method.

To add more than one item to a set use the `update()` method.

Example

Add an item to a set, using the `add()` method:

```
thisset = {"apple", "banana", "cherry"}

thisset.add("orange")

print(thisset)
```

NOTE: Lists cannot be added to a set as elements because Lists are not hashable whereas Tuples can be added because tuples are immutable and hence Hashable.

Removing elements from the Set

Using remove() method or discard() method:

- Elements can be removed from the Set by using the built-in remove()function but a KeyError arises if the element doesn't exist in the set.
- To remove elements from a set without KeyError, use discard(), if the element doesn't exist in the set, it remains unchanged.

Note: If the set is unordered then there's no such way to determine which element is popped by using the pop() function.

Python Set in -built functions:

Method	Description
add()	Adds an element to the set
clear()	Removes all the elements from the set
copy()	Returns a copy of the set
difference()	Returns a set containing the difference between two or more sets
difference_update()	Removes the items in this set that are also included in another, specified set
discard()	Remove the specified item
intersection()	Returns a set, that is the intersection of two other sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)
isdisjoint()	Returns whether two sets have a intersection or not
issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set
remove()	Removes the specified element
symmetric_difference()	Returns a set with the symmetric differences of two sets
symmetric_difference_update()	inserts the symmetric differences from this set and another
union()	Return a set containing the union of sets
update()	Update the set with the union of this set and others

Examples:

S1={1,2,3,4,5}

S1.remove(2)

print(S1)

Output:

{1,3,4,5}

S1={1,2,3,4}

S2={1,2,3,4,5}

print(S1.issubset(S2))

Output:

True

print(S2.issuperset(S1))

Output:

True

>>>

S1={1,2,3,4,5}

S2={5,6,7}

print(S1.union(S2)) #

print(S1|S2)

Output:

{1,2,3,4,5,6,7}

>>>

print(S1.intersection(S2)) #

print(S2&S2)

Output:

{5}

>>>

print(S1.difference(S2))

print(S1-S2)

Output:

{1,2,3,4}

```
>>>
print(S1.symmetric_difference(S2))
print(S1^S2)
```

Output:
{1,2,3,4,6,7}

3.10 List manipulating method

SN	Function	Description
1	list.append(obj)	The element represented by the object obj is added to the list.
2	list.clear()	It removes all the elements from the list.
3	List.copy()	It returns a shallow copy of the list.
4	list.count(obj)	It returns the number of occurrences of the specified object in the list.
5	list.extend(seq)	The sequence represented by the object seq is extended to the list.
6	list.index(obj)	It returns the lowest index in the list that object appears.
7	list.insert(index, obj)	The object is inserted into the list at the specified index.
8	list.pop(obj=list[-1])	It removes and returns the last object of the list.
9	list.remove(obj)	It removes the specified object from the list.
10	list.reverse()	It reverses the list.
11	list.sort([func])	It sorts the list by using the specified compare function if given.

Table 3.7 List Buit-in Methods

Lecture 17. Python Dictionary and its manipulation method

3.11 Dictionary in Python

It is a collection of keys values, used to store data values like a map, which, unlike other data types which hold only a single value as an element.

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

3.11.1 Properties of Dictionary Keys

There are two important points to remember about dictionary keys −

More than one entry per key is not allowed. Which means no duplicate key is allowed. When duplicate keys are encountered during assignment, the last assignment wins.

Keys must be immutable. Which means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed.

3.11.2 Accessing value to a dictionary

In order to access the items of a dictionary refer to its key name. Key can be used inside square brackets. . Following is a simple example −

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

print("dict['Name']:",dict['Name'])

print("dict['Age']: ", dict['Age'])
```

When the above code is executed, it produces the following result −

```
dict['Name']:Zara

dict['Age']: 7
```

There is also a method called **get()** that will also help in accessing the element from a dictionary.This method accepts key as argument and returns the value.

```
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}
print("Accessing aelement using get:")
print(Dict.get(3))
```

3.11.3 Updating Dictionary

You can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example –

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
```

```
{'brand': 'Ford', 'model':
'Mustang', 'year': 1964,
'color': 'White'}
```

```
car.update({"color": "White"})
```

print(car)Deleting Elements using del

Keyword

The items of the dictionary can be deleted by using the del keyword

```
Dict = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}

print("Dictionary =")
print(Dict)

#Deleting some of the Dictionar data
del(Dict[1])

print("Data after deletion Dictionary=")
print(Dict)
```

Output

Dictionary ={1: 'Geeks', 'name': 'For', 3: 'Geeks'}
Data after deletion
Dictionary={'name': 'For', 3: 'Geeks'}

3.12 Built-in Dictionary Functions & Methods

Python includes the following dictionary functions –

Sr.No.	Function with Description

1	**cmp(dict1, dict2)** Compares elements of both dict.
2	**len(dict)** Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
3	**str(dict)** Produces a printable string representation of a dictionary
4	**type(variable)** Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Figure 3.8 Dictionary Functions

3.13 Dictionary methods

Method	Description
dict.clear()	Remove all the elements from the dictionary
dict.copy()	Returns a copy of the dictionary
dict.get(key, default = "None")	Returns the value of specified key
dict.items()	Returns a list containing a tuple for each key value pair
dict.keys()	Returns a list containing dictionary's keys
dict.update(dict2)	Updates dictionary with specified key-valuepairs
dict.values()	Returns a list of all the values of dictionary
pop()	Remove the element with specified key
popItem()	Removes the last inserted key-value pair
dict.setdefault(key,default= "None")	set the key to the default value if the

key isnot specified in the dictionary

dict.has_key(key)	returns true if the dictionary contains thespecified key.
dict.get(key, default = “None”)	used to get the value specified for the passedkey.

Lecture 18. Function

3.15 Function.

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

Defining a Function

You can define functions to provide the required functionality. Here are simple rules to define a function in Python.

- Function blocks begin with the keyword **def** followed by the function name and parentheses (( )).
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon (:) and is indented.
- The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

Program for sum of digits from range x to y def sum(x,y): –>

Master code

```
def sum(x,y):

s=0

for i in range(x,y+1):
    s-s+i

print("sum of integers from x to y :",s)
```

sum(50,75) –> **Driver code**

Output:

sum of integers from x to y : 1625

Function calling is called driver code and function definition with block of statements is called master code

Syntax

```
def functionname( parameters ):
   "function_docstring"
   function_suite
   return [expression]
```

By default, parameters have a positional behavior and you need to inform them in the same order that they were defined.

Example

The following function takes a string as input parameter and prints it on standard screen.

```
def printme( str ):
   "This prints a passed string into this function"
   print str
   return
```

3.16 Differentiate between argument and parameter.

Ans: Parameters are of two types:

- Formal parameter
- Actual Parameter

Formal parameters are those written in function definition and actual parameters are those written in function calling.

Formal parameters are called argument.
Arguments are the values that are passed to function definition.

Example:

```
Def fun(n1):
   Print(n1)
a=10
fun(a)
```

here n1 is argument and a is parameter.

3.17 Different types of arguments inpython.

Ans:

You can call a function by using the following types of formal arguments −

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Required arguments

Required arguments are the arguments passed to a function in correct positional order. Here, the number of arguments in the function call should match exactly with the function definition.

To call the function *printme()*, you definitely need to pass one argument, otherwise it gives a syntax error as follows −

```
def printme(str):
    print (str)

printme()# error missing 1 required positional

argument def printme( ):

    print(str)

printme('hello') #error, printme() takes 0 positional
arguments but 1 was given
```

```
def printme(str):
    print(str)

printme('abc')
```

```
Ou
tp
ut
```

Keyword arguments

Keyword arguments are related to the function calls. When you use keyword arguments in a function call, the caller identifies the arguments by the parameter name.

This allows you to skip arguments or place them out of order because the Python interpreter is able to use the keywords provided to match the values with parameters. You can also make keyword calls to the *printme()* function in the following ways −

```
def printinfo( name, age ):
   "This prints a passed info into this function"
   print "Name: ", name
   print "Age ", age
   return;

# Now you can call printinfo function
printinfo( age=50, name="miki" )
```

When the above code is executed, it produces the following result −

```
Name:  miki
Age  50
```

Default arguments

A default argument is an argument that assumes a default value if a value is not provided in the function call for that argument. The following example gives an idea on default arguments, it prints default age if it is not passed −

```
def printinfo( name, age = 35 ):
   "This prints a passed info into this function"
   print "Name: ", name
   print "Age ", age
   return;

# Now you can call printinfo function
printinfo( age=50, name="miki" )
printinfo( name="miki" )
```

When the above code is executed, it produces the following result −

```
Name:  miki
Age  50
Name:  miki
Age  35
```

def display(a='hello',b='world'): # you can have only default arguments

print(a,b)

display()

Output:

hello world

```
def display(a,c=10,b): # default argument will always come as last argument
    d=a+b+c
    print(d)
display(2,3) # error, non default argument follows default argument
```

```
def display(a,b,c=10):
    d=a+b+c
    print(d)
display(2,3)
```

Output:

15

Variable-length arguments

You may need to process a function for more arguments than you specified while defining the function. These arguments are called *variable-length* arguments and are not named in the function definition, unlike required and default arguments.

Syntax for a function with non-keyword variable arguments is this −

```
def functionname([formal_args,] *var_args_tuple ):
   "function_docstring"
   function_suite
   return [expression]
```

An asterisk (*) is placed before the variable name that holds the values of all nonkeyword variable arguments. This tuple remains empty if no additional arguments are specified during the function call. Following is a simple example −

Variable length with first extra argument:

```
def printinfo( arg1, *vartuple ):
   "This prints a variable passed arguments"
   print "Output is: "
   print arg1
   for var in vartuple:
      print var
   return;

# Now you can call printinfo function
printinfo( 10 )
printinfo( 70, 60, 50 )
```

When the above code is executed, it produces the following result −

```
Output is:
10
Output is:
70
60
50
```

variable length argument Example:

```
def display(*arg):
    for i in arg:
        print(i)
display('hello','world')
```

Output:

hello

world

3.18 Return statement:

The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

All the above examples are not returning any value. You can return a value from a function as follows −

```
def sum( arg1, arg2 ):
   # Add both the parameters and return them."
   total = arg1 + arg2
   print "Inside the function : ", total
   return total;

# Now you can call sum function
total = sum( 10, 20 );
print "Outside the function : ", total
```

When the above code is executed, it produces the following result −

```
Inside the function :  30
Outside the function :  30
```

The Return Statement: The return statement is used to return a value from the function to a calling function. It is also used to return from a function i.e. break out of the function.

Example:

Program to return the minimum of two numbers:

*Returning multiple values***:** It is possible in python to return multiple values def

compute(num):

return num*num,num**3

 # returning multiple value

square,cube=compute(2)

print("square = ",square,"cube =",cube)

Lecture 19. Anonymous function

3.19 Anonymous Function

The *Anonymous* Functions

These functions are called anonymous because they are not declared in the standard manner by using the *def* keyword. You can use the *lambda* keyword to create small anonymous functions.

- Lambda forms can take any number of arguments but return just one value in the form of an expression. They cannot contain commands or multiple expressions.
- An anonymous function cannot be a direct call to print because lambda requires an expression
- Lambda functions have their own local namespace and cannot access variables other than those in their parameter list and those in the global namespace.
- Although it appears that lambda's are a one-line version of a function, they are not equivalent to inline statements in C or C++, whose purpose is by passing function stack allocation during invocation for performance reasons.

Syntax

The syntax of *lambda* functions contains only a single statement, which is as follows −

```
lambda [arg1 [,arg2,.....argn]]:expression
```

```
sum = lambda arg1, arg2: arg1 + arg2;

# Now you can call sum as a function
print "Value of total : ", sum( 10, 20 )
print "Value of total : ", sum( 20, 20 )
```

When the above code is executed, it produces the following result −

```
Value of total :  30
Value of total :  40
```

Program to find cube of a number using lambda function

```
cube=lambda x:
x*x*x
print(cube(2))
```

Output:

8

Note:

- The statement cube=lambda x: x*x*x creates a lambda function called cube, which takes a single argument and returns the cube of a number.
- Lambda function does not contain a return statements
- It contains a single expression as a body not a block of statements as a body

Q6. Define scope of a variable. Also differentiate local and globalvariable.

Ans:

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python −

- Global variables
- Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example

```
total = 0; # This is global variable.
# Function definition is here
def sum( arg1, arg2 ):
   # Add both the parameters and return them."
   total = arg1 + arg2; # Here total is local variable.
   print "Inside the function local total : ", total
   return total;

# Now you can call sum function
sum( 10, 20 );
print "Outside the function global total : ", total
```

When the above code is executed, it produces the following result −

```
Inside the function local total :  30
Outside the function global total :  0
```

1. Program to access a local variable outside a functions

```
def demo():
  q=10
  print("the value of local variable is",q)
demo()
print("the value of local variable q is:",q)
```

#error, accessing a local variableoutside the scope will cause an error

Output:

the value of local variable is 10

Traceback (most recent call last):

File "C:/Users/Students/AppData/Local/Programs/Python/Python38-32/vbhg.py", line 5, in <module>

print("the value of local variable q is:",q)

NameError: name 'q' is not defined

2. Program to read global variable from a local scope

```
def demo():
        print(s)

s=’I love python’
```

```
demo()
```

Output:

I love python

3. Local and global variables with the same name

```
def demo():
    s='I love
    pythonprint(s)
s='I love programming'
demo() # first function is called, after that other satements print(s)
```

Output:

I love python

I love programming

The Global Statement

Global statement is used to define a variable defined inside a function as a global variable. To make local variable as global variable, use global keyword

Example:

```
a=20
def display():
    global a
    a=30
    print('value of a is',a)
display()
print('the value of an outside function is',a)
```

Output:

value of a is 30

the value of an outside function is 30

Note:

Since the value of the global variable is changed within the function, the value of 'a' outside the function will be the most recent value of 'a'

Lecture 20. Recursion Function

3.20 Recursion-

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

Example

Recursion Example

```
def tri_recursion(k):
  if(k>0):
    result = k+tri_recursion(k-1)
    print(result)
  else:
    result = 0
  return result

print("\n\nRecursion Example Results")
tri_recursion(6)
```

Q Program to find the factorial of a number using recursion

```
def factorial(n):
        if n==0: # base condition
                return 1
        return n * factorial(n-1)
print(factorial(5))
```

Output:

120

Note: Recursion ends when the number n reduces to 1. This is called base condition. Every recursive function must have a base condition that stops the recursion or else the function calls itself infinitely.

Q8. Program to find the fibonacci series using recursion.

Ans:

```
def fibonacci(n):
  if(n <= 1):
```

```
        return n
    else:
        return(fibonacci(n-1) + fibonacci(n-2))

n = int(input("Enter number of terms:"))

print("Fibonaccisequence:")
for i in range(n):
    print(fiboncci(i)
    ,
```

Q9. Program to find the GCD of two numbers using recursion.

Ans:

```
def gcd(a,b):
    if(b==0):
        returna
    else:
        return gcd(b,a%b)
a=int(input("Enter first number:"))
b=int(input("Enter second number:"))
GCD=gcd(a,b)
print("GCD is: )
print(GCD)
```

Q10. Program to find the sum of elements in a list recursively.

Ans:

```
def sum_arr(arr,size):
if (size == 0):
  return 0
else:
    return arr[size-1] + sum_arr(arr,size-1)
  n=int(input("Enter the numberof elements for list:"))
  a=[]
```

```
for i in range(0,n):
   element=int(input("Enter element:"))
   a.append(element)
print("The list is:")
print(a)
   print("Sum of items in list:")
   b=sum_arr(a,n)
   print(b)
```

Q11. Program to check whether a string is a palindrome or not using recursion. Ans:

```
def is_palindrome(s):
    if len(s) < 1:
        return True
  else:
      if s[0] == s[-1]:
      return is_palindrome(s[1:-1])
     else:
           return False
 a=str(input("Enter string:"))
if(is_palindrome(a)==True):
   print("String is a palindrome!")
else:
  print("String isn't a palindrome!")
```

Important Ques Unit-3

i. Explain mutuable sequences in python .(2019-20,2023-24)

ii. Compare list and tuple with suitable example. Explain the concept of list comprehension.(2019-20)

iii. Write a python program that accepts a sentence and calculate the number of digits. Uppercase and lowercase letters. (2021-22)

iv. Write a program to print fibonnacci series upto n terms using user-defined function (2021-22)

v. Explain output of following function . (2022-23)

```
def printalpha(abc_list , num_list):
for char in abc_list:
   for num in num_list:
print(char, num)
return
printalpha(['a','b','c'],[1,2,3])
```

Unit-4
File handling in Python
Lecture 21. File Operations: Reading and Writing Files.

4.1 File handling in Python- Python supports file handling and allows users to handle files i.e., to read and write files, along with many other filehandling options, to operate on files.

The concept of file handling has stretched over various other languages, but the implementation is either complicatedor lengthy, like other concepts of Python, this concept here is also easy and short. Python treats files differently as textor binary and this is important. Each line of code includes a sequence of characters, and they form a text file. Each lineof a file is terminated with a special character, called the EOL or End of Line characters like comma {,} or newline character. It ends the current line and tells the interpreter a new one has begun.

4.1.1 Advantages of File Handling in Python

- **Versatility**: File handling in Python allows you to perform a wide range of operations, such as creating, reading,writing, appending, renaming, and deleting files.
- **Flexibility**: File handling in Python is highly flexible, as it allows you to work with different file types (e.g. text files, binary files, CSV files, etc.), and to perform different operations on files (e.g. read, write, append, etc.).
- **User–friendly**: Python provides a user-friendly interface for file handling, making it easy to create, read, andmanipulate files.
- **Cross-platform**: Python file-handling functions work across different platforms (e.g. Windows, Mac, Linux),allowing for seamless integration and compatibility.

4.1.2 Disadvantages of File Handling in Python

- **Error-prone:** File handling operations in Python can be prone to errors, especially if the code is not carefullywritten or if there are issues with the file system (e.g. file permissions, file locks, etc.).
- **Security risks**: File handling in Python can also pose security risks, especially if the program accepts user input thatcan be used to access or modify sensitive files on the system.
- **Complexity**: File handling in Python can be complex, especially when working with more advanced file formats or operations. Careful attention must be paid to the code to ensure that files are handled properly and securely.
- **Performance**: File handling operations in Python can be slower than other programming languages, especiallywhen dealing with large files or performing complex operations.

Python provides built-in functions and methods to perform various file operations like reading, writing, and updatingfiles

4.2 Python File Open

Before performing any operation on the file like reading or writing, first, we have to open that file. For this, we should use Python's inbuilt function open() but at the time of opening, we have to specify the mode, which represents the purpose of the opening file.

f = open(filename, mode)

Eg:- file = open('example.txt', 'r') # Opens the file in read mode

The mode argument is a string that specifies the mode in which the file is opened:

'r' for reading (default)

'w' for writing (creates a new file or truncates the file first)
'x' for exclusive creation (fails if the file already

exists) 'a' for appending (creates a new file if it

does not exist)'b' for binary mode

't' for text mode (default)

'+' for updating (reading and writing)

r+: To read and write data into the file. The previous data in the file will be

overridden.w+: To write and read data. It will override existing data.

a+: To append and read data from the file. It won’t override existing data.

Lecture 22. Discussed about How use read functions like read(), readline(), readlines().

4.3 Reading from a File

Once the file is opened, you can read its content using methods like **read(), readline(), or readlines():**

 # Read the entire file

content = file.read()

print(content)

 # Read one line at a time

line = file.readline()

print(line)

 # Read all lines into a list

lines = file.readlines()

print(lines)

file.close() # Always close the file when you're done with it

Q:-Python code to illustrate read() mode

file = open("cse.txt", "r")
print (file.read())

Output:

Hello world

Welcome CSE

123 456

Q:- In this example, we will see how we can read a file using the with statement in Python.

Python code to illustrate with

with open("geeks.txt") as file:

data = file.read()

print(data)

Q:- Another way to read a file is to call a certain number of characters like in the following code the interpreter willread the first five characters of stored data and return it as a string:

```
# Python code to illustrate read() mode character

wisefile = open("geeks.txt", "r")

print (file.read(5))
```

Q:-We can also split lines while reading files in Python. The split() function splits the variable when space isencountered. You can also split using any characters as you wish.

```
# Python code to illustrate split() function

with open("geeks.txt", "r") as file:

data = file.readlines()

for line in data:

 word = line.split()

 print (word)
```

OUTPUT:-

```
['Hello', 'world']
['CSE', 'Welcome']
['123', '456']
```

Lecture 23. Writing files in Python

4.4 Writing to a File

To write to a file, you need to open it in a write ('w'), append ('a'), or update ('+') mode. Then, use the write() orwritelines() methods:

Syntax:-

```
file = open('example.txt', 'w')  # Open the file in write mode
file.write('Hello, World!\n')  # Write a string to the file
lines = ['First line.\n', 'Second line.\n']
file.writelines(lines)  # Write a list of strings to the file
file.close()
```

Closing a File

It's important to close the file when you're done with it to free up system resources. Use the close() method: file.close()

Using with Statement

The with statement simplifies file handling by automatically taking care of closing the file once it leaves the with block,even if exceptions occur:

Eg:-

```
with open('example.txt', 'r') as file:

 content = file.read()

print(content)

# File is automatically closed here
```

Working of Append Mode

```
# Python code to illustrate append()

modefile = open('geek.txt', 'a')
file.write("This will add this line")
file.close()
```

4.5 Implementing all the functions in File Handling

```
import os
def create_file(filename):
   try:
      with open(filename, 'w') as f:
         f.write('Hello, world!\n')
      print("File " + filename + " created successfully.")
 except IOError:
      print("Error: could not create file " + filename)

def read_file(filename):
```

```
    try:
        with open(filename, 'r') as f:
            contents = f.read()
            print(contents)
    except IOError:
        print("Error: could not read file " + filename)

def append_file(filename, text):
    try:
        with open(filename, 'a') as f:
            f.write(text)
        print("Text appended to file " + filename + "
    successfully.")except IOError:
        print("Error: could not append to file " + filename)

def rename_file(filename, new_filename):
    try:
        os.rename(filename, new_filename)
        print("File " + filename + " renamed to " + new_filename + "
    successfully.")except IOError:
        print("Error: could not rename file " + filename)

def delete_file(filename):
    try:
        os.remove(filename)
        print("File " + filename + " deleted
    successfully.")except IOError:
        print("Error: could not delete file " + filename)


if __name__== '_main_':
    filename = "example.txt"
    new_filename = "new_example.txt"

    create_file(filename)
    read_file(filename)
    append_file(filename, "This is some additional text.\n")
    read_file(filename)
    rename_file(filename, new_filename)
    read_file(new_filename)
    delete_file(new_filename)
```

Output:

```
File example.txt created successfully.
Hello, world!
```

Text appended to file example.txt successfully.
Hello, world!
This is some additional text.
File example.txt renamed to new_example.txt successfully.
Hello, world!
This is some additional text.
File new_example.txt deleted successfully.

Handling File Paths

For file paths, you can use raw strings (e.g., r'C:\path\to\file.txt') to avoid escaping backslashes in Windows paths, or use forward slashes which Python understands across platforms (e.g., 'C:/path/to/file.txt' or './folder/file.txt').

Lecture 24. Manipulating File Pointer

4.6 File Pointer Manipulation- In Python, you can manipulate the file pointer, which represents the current position in the file where the next read orwrite operation will occur. Python provides the seek() method to move the file pointer to a specific position within thefile.

Syntax:-

f.seek(offset, from_what), where f is file pointer

The seek() method takes two arguments:

1. **offset:** The number of bytes to move the file pointer. Positive values move the pointer forward, negative values move it backward.
2. **from_what:** This argument specifies the reference point for the offset. It can take one of three values:

- 0 (default): The beginning of the file.
- 1: The current file position.
- 2: The end of the file.

By default from_what argument is set to 0.

Note: Reference point at current position / end of file cannot be set in text mode except when offset is equal to 0.

Example:-

Example 1: Let's suppose we have to read a file named "CSE.txt" which contains the following text:"Code is like humor. When you have to explain it, it's bad."

```
f = open("CSE.txt", "r")

# Second parameter is by default 0
# sets Reference point to twentieth index position from the beginning
f.seek(20)

# prints current position
print(f.tell())

print(f.readline())
f.close()
```

Output:

20

When you have to explain it, it's bad.

Example 2: Seek() function with negative offset only works when file is opened in binary mode. Let's suppose thebinary file contains the following text.

b'Code is like humor. When you have to explain it, its bad.'

```
f = open("data.txt", "rb")
```

```
# sets Reference point to tenth#
position to the left from end
f.seek(-10, 2)

# prints current position
print(f.tell())

# Converting binary to string and
# printing
print(f.readline().decode('utf-8'))

f.close()
```

Output:

```
47
, its bad.
```

Example3:- seek() method to manipulate the file pointer:

```
# Open a file in read mode
file = open('example.txt', 'r')

# Move the file pointer to the 10th byte from the beginning
file.seek(10)

# Read the content from the current position
content = file.read()
print(content)

# Move the file pointer 5 bytes backward from the current position
file.seek(-5, 1)

# Read the content from the current position
content = file.read()
print(content)

# Move the file pointer to the end of the file
file.seek(0, 2)

# Write content at the end of the file
file.write("\nAdding content at the end.")

# Close the
filefile.close()
```

In this example:

The first seek(10) call moves the file pointer to the 10th byte from the beginning of the file.
The second seek(-5, 1) call moves the file pointer 5 bytes backward from the current position.
The third seek(0, 2) call moves the file pointer to the end of the file.
It's important to note that seeking beyond the end of the file when writing will cause the file to be extended withnull bytes up to the specified position. Therefore, you can seek beyond the end of the file when writing to appenddata. However, seeking beyond the end of the file when reading will result in an error.

Lecture 25. File of Operations

4.7 File of Operations-Using a file of operations typically involves reading from or writing to a file that contains a series of instructions or data manipulations. These operations can vary widely depending on the context, such as programming, data processing, or system administration. Let's break down how to approach this in a few different scenarios:

4.7.1 Programming and Scripting

In programming, a file of operations might contain a list of commands or function calls that should be executed sequentially. Here's how you might approach this:

Reading the File:
Open the file in the appropriate mode (read, write, append, etc.).
Read the file line by line or as a whole, depending on your needs.

Parsing the Operations:
For each line or operation, parse the instruction. This could involve simple string manipulation or more complex parsing if the instructions are in a specific format (JSON, XML, etc.).
It's essential to implement error handling to deal with malformed instructions or unsupported operations.

Executing the Operations:
Execute each parsed operation. This might involve calling functions, executing system commands, or performing file manipulations.
Ensure that operations are executed in the correct order and handle any dependencies between operations.

4.7.2 Data Processing

In data processing, a file of operations might list data transformations or analyses to perform on a dataset. This could involve:

Reading and Writing Data:
Use appropriate libraries for data manipulation (e.g., pandas in Python) to read your dataset.
Read the operations file to determine what transformations or analyses need to be performed.

Performing Operations:
For each operation, apply the corresponding data transformation. This might involve filtering data, performing calculations, or aggregating information.
After performing an operation, you might need to write the result to a file or prepare it for the next operation.

4.7.3 System Administration

For system administration, a file of operations might contain a list of system commands or configuration changes to apply to a server or network of computers.

Automating Tasks:

Use a scripting language suitable for system administration (such as Bash for Linux/Unix systems or PowerShell for Windows).

Read each operation from the file and use the script to execute the corresponding system command or change the system configuration.

Ensuring System Integrity:

Implement checks to verify that each operation completes successfully.

Log the outcomes of operations for audit purposes and to troubleshoot any issues that arise.

General Tips

Error Handling: Always include error handling to manage unexpected or malformed operations gracefully.

Logging: Keep a log of operations performed and any errors or warnings generated. This is invaluable for debugging and verifying that operations have been executed correctly.

Security: Be cautious when executing operations from a file, especially if the operations involve system commands or could impact data integrity. Validate and sanitize the operations to prevent security vulnerabilities.

By breaking down the process into manageable steps and considering the context in which you're using the file of operations, you can effectively implement and automate a wide range of tasks.

Above Operation in case of Python Language

1. Programming and Scripting

Suppose you have a file named **operations.txt** with lines of operations like:

print,Hello World!
add,3,5

You want to read these operations and execute them. Let's assume add operation sums the numbers.def

```
handle_print(args):
    print(*args)

def handle_add(args):
    result = sum(map(int, args))
    print(result)

# Mapping operation names to functions
operations = {
    'print': handle_print,
    'add': handle_add,
}
```

```
with open('operations.txt', 'r') as file:
    for line in file:
        op, *args = line.strip().split(',')
        if op in operations:
            operations[op](args)
        else:
            print(f"Unsupported operation: {op}")
```

2. Data Processing

Imagine you have a CSV file data.csv and an operations file data_operations.txt that contains operations like filter, >10 and average. For simplicity, let's handle a single operation: filtering values greater than 10.

Program:-

```
import pandas as pd
# Assuming a simple CSV file with one column of integers
df = pd.read_csv('data.csv')

with open('data_operations.txt', 'r') as file:
    for line in file:
        operation, value = line.strip().split(',')
        if operation == 'filter':
            df = df[df['column_name'] > int(value)]

# Assuming you might want to print or save the filtered data
print(df)
```

3. System Administration

For system administration, let's automate the creation of backup files. Assume an operations file **system_ops.txt** with content like backup,/path/to/file.

```
import shutil
import os

def backup_file(file_path):
    if os.path.exists(file_path):
        shutil.copy(file_path,
        f"{file_path}.backup")print(f"Backup
        created for {file_path}")
    else:
        print(f"File does not exist: {file_path}")

with open('system_ops.txt', 'r') as file:
    for line in file:
        operation, path = line.strip().split(',')
        if operation == 'backup':
```

```
backup_file(path)
```

General Python Tips for Handling Files of Operations

Use Context Managers: Always use with open(...) as ... for safely opening and closing files.

Error Handling: Wrap operations in try/except blocks to gracefully handle exceptions.

Dynamic Function Mapping: As shown in the programming example, map operation names to functions for a cleanand scalable way to execute operations.

Validate Inputs: Especially when executing system operations or modifying data, validate and sanitize inputs to prevent errors or security issues.

(Examples) :- Python Program (Fibonacci Series)

Program:- Discussed about Fibonacci Series of Program.

The Fibonacci series is a sequence of numbers where each number is the sum of the two preceding ones, usually starting with 0 and 1. That is, the sequence starts 0, 1, 1, 2, 3, 5, 8, 13, 21, and so on. It's a classic example used in programming to demonstrate various coding techniques, including recursion, iteration, and dynamic programming.

1. Recursive Approach

The recursive approach directly implements the mathematical definition of the Fibonacci sequence. However, it's not efficient for large numbers due to repeated calculations and a high recursive call stack usage.

```
def fibonacci_recursive(n):
   if n <= 0:
         return 0
     elif n == 1:
         return 1
   else:
      return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)

# Example usage:
n = 10
print(f"The {n}th Fibonacci number (recursive) is: {fibonacci_recursive(n)}")
```

2. Iterative Approach

The iterative approach uses a loop to calculate the Fibonacci numbers up to n. This method is much more efficient than recursion for large numbers.

```
def fibonacci_iterative(n):
   a, b = 0, 1
   for _ in range(n):
      a, b = b, a + b
   return a

# Example usage:
n = 10
print(f"The {n}th Fibonacci number (iterative) is: {fibonacci_iterative(n)}")
```

Python Program(Matrix Addition)

Program: - Discussed about Addition of two Matrix of Program.

Adding two matrices in Python can be done in various ways, ranging from basic loops to using sophisticated libraries like NumPy, which is designed for scientific computing and can handle operations on large multi-dimensional arrays and matrices efficiently**.**

```
def add_matrices(matrix1, matrix2):
    # Ensure the matrices have the same dimensions
    if len(matrix1) != len(matrix2) or len(matrix1[0]) !=
        len(matrix2[0]):return "Matrices are of different sizes."

    # Create a new matrix to store the result
    result_matrix = [[0 for _ in range(len(matrix1[0]))] for _ in range(len(matrix1))]

    # Iterate over the matrices to add corresponding elements
    for i in range(len(matrix1)):
        for j in range(len(matrix1[0])):
            result_matrix[i][j] = matrix1[i][j] + matrix2[i][j]

    return result_matrix
# Example usage
matrix1 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
matrix2 = [[9, 8, 7], [6, 5, 4], [3, 2, 1]]

print("Resultant Matrix:")
for row in add_matrices(matrix1, matrix2):
    print(row)
```

Python Program(Transpose Matrix of Program.)

Program:-

Transposing a matrix means flipping a matrix over its diagonal, turning the matrix's rows into columns and columns into rows. This operation is common in mathematics and programming, especially in the context of linear algebra and data manipulation.

Basic Python Loops: Good for educational purposes and environments where external dependencies are discouraged or not allowed. However, manually coding the transpose operation can be error-prone for complex data structures.
Using NumPy: Offers a simple, efficient, and less error-prone method for transposing matrices. Highly recommended for scientific computing, data analysis, and any application requiring manipulation of large numerical datasets.

Program(Using Python Loops)

```
def transpose_matrix(matrix):
   # Initialize the transposed matrix with zeros
   transposed = [[0 for _ in range(len(matrix))] for _ in range(len(matrix[0]))]

   # Iterate through rows
   for i in range(len(matrix)):
      # Iterate through columns
      for j in
      range(len(matrix[i])):
         # Assign transposed values
         transposed[j][i] =
         matrix[i][j]

   return transposed

# Example usage
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
transposed_matrix = transpose_matrix(matrix)

print("Original
Matrix:")for row in
matrix:
   print(row)

print("\nTransposed Matrix:")
for row in transposed_matrix:
   print(row)
```

Python Program (Identity Matrix)

Program:- Python identity Matrix of Program.

```
def identity_matrix(n):
   """Create an n x n identity matrix."""
   return [[1 if i == j else 0 for j in range(n)] for i in range(n)]

# Example usage
n = 4
```

```
print("Identity Matrix of size",
n)for row in identity_matrix(n):
   print(row)
```

Python Program (Multiplication table)

Python Program:- Python program to Print any no of table Program.

```
def print_multiplication_table(number, range_limit):
   """
   Prints the multiplication table for a given number up to a specified range.

   Parameters:
   number (int): The number for which the multiplication table is to be printed.
   range_limit (int): The range up to which the table should be printed.
   """
   for i in range(1, range_limit + 1):
      print(f"{number} x {i} = {number *
      i}")

def main():
   # Prompt the user for input
   number = int(input("Enter the number for the multiplication table: "))
   range_limit = int(input("Enter the range up to which to print the table: "))

   # Print the multiplication table
   print(f"\nMultiplication table for {number} up to
   {range_limit}:")print_multiplication_table(number, range_limit)

if __name__== "_main_":
   main()
```

Python Program (Leap Year)

Program:- Python Program how to check given year is leap or not

```
def is_leap_year(year):
   """
   Returns True if the given year is a leap year, False otherwise.
   """
   # Year is divisible by 4
   if year % 4 == 0:
      # Year is not divisible by 100 unless it's also divisible by 400
      if year % 100 == 0:
         if year % 400 == 0:
            return True
         else:
            return False
```

```
        else:
            return True
    else:
        return False

def main():
    year = int(input("Enter a year: "))
    if is_leap_year(year):
        print(f"{year} is a leap
    year.")else:
        print(f"{year} is not a leap year.")

if __name__== " _main _":
    main()
```

Python Program (Perfect Number)

Program:- Python Perfect number

program

A perfect number is a positive integer that is equal to the sum of its proper divisors, excluding itself. For example, 28 is a perfect number because its divisors are 1, 2, 4, 7, and 14, and
1+2+4+7+14=28

Below is a Python program that checks if a given number is a perfect number. The program defines a function to calculate the sum of divisors of a number and then checks if this sum equals the number itself.

```
def is_perfect_number(number):
    if number < 1:
        return False

    sum_of_divisors = 0
    # Check for divisors of the number
    for possible_divisor in range(1, number):
        if number % possible_divisor == 0:
            sum_of_divisors += possible_divisor

    # Compare the sum of divisors (excluding the number itself) to the number
    return sum_of_divisors == number

def main():
    number = int(input("Enter a number: "))
    if is_perfect_number(number):
        print(f"{number} is a perfect number.")
    else:
        print(f"{number} is not a perfect number.")

if __name___== "_main___":
```

main()

Python Program (Armstrong Number)

Program:- Python Armstrong number

program

An Armstrong number (also known as a narcissistic number) is a number that is equal to the sum of its own digits each raised to the power of the number of digits. For example, 153 is an Armstrong number because it has 3 digits, and $1^3 + 5^3 + 3^3 = 153$

To write a Python program that checks if a given number is an Armstrong number, you need to:

- Get the number of digits in the number.
- Calculate the sum of the digits each raised to the power of the number of digits.
- Compare the sum to the original number.

Here's a **Python program that performs these steps:**

```
def is_armstrong_number(number):
    # Convert the number to a string to easily iterate over its digits
    str_number = str(number)
    # Calculate the number of digits
    num_digits = len(str_number)

    # Calculate the sum of digits raised to the power of the number of digits
    sum_of_powers = sum(int(digit) ** num_digits for digit in str_number)

    # Compare the sum to the original number
    return sum_of_powers == number

def main():
    number = int(input("Enter a number: "))
    if is_armstrong_number(number):
        print(f"{number} is an Armstrong
    number.")else:
        print(f"{number} is not an Armstrong number.")

if __name__== "_main_":
    main()
```

Important Ques -CO4

i. Discuss various file opening mode in python (2020-21)

ii. Discuss Exceptions and Assertions in Python. Explain with a suitable example. Explain any two built-in exceptions.(2021-22, 2019-20)

iii. There is a file named Input .Txt. Enter some positive numbers into the file named Input.Txt .Read the contents of thefile and if it is an odd number write it to ODD.Txt and if the number is even , write it to EVEN,Txt. (2021-22)

iv. What are file input and output operations in python programming? (2019-20, 2020-21)

v. Write a python program to read the content of a file and then create a file COUNT.Txt to write the numbers of lettersand digits of the readed content from the given file (2022-23)

Unit – 5

Introduction to Python Packages

Lecture 26. Python Packages

5.1 Python Packages- Python packages are a way of organizing and distributing Python code. They allow you to bundle multiple files or modules into a single, importable package, making code reuse and distribution easier. Python packages can include libraries, frameworks, or collections of modules that provide additional functionality or enable specific tasks. Here's anoverview of key concepts and steps related to Python packages:

Key Concepts

Module: A Python file containing Python definitions, statements, and functions. It's the smallest unit of code reuse in Python.

Package: A way of collecting related modules together within a single tree-like hierarchy. Very complex packages mayinclude subpackages at various levels of the hierarchy.

Installing Packages

To install packages, you generally use pip, Python's package installer

Commonly Used Packages

Here are some widely used Python packages for various purposes:

NumPy: Numerical computing and array operations.

Pandas: Data analysis and manipulation.

Requests: HTTP requests for humans.

5.2 Matplotlib

Matplotlib is a popular Python library used for creating static, animated, and interactive visualizations in Python. It provides a wide range of plotting tools and features for creating high-quality graphs, charts, and plots.

Here are some of the key components and functions provided by Matplotlib:

Key Components:

Figure: The entire window or page where the plots and charts are drawn.

Axes: The area where data is plotted. It includes the X-axis, Y-axis, and the plot itself.

Axis: The X-axis and Y-axis, which define the scale and limits of the plot.

Artist: Everything that is drawn on the figure (such as lines, text, shapes) is an artist.

Installation:-

python -m pip install -U matplotlib

Matplotlib

Matplotlib is an amazing visualization library in Python for 2D plots of arrays. Matplotlib is a multi-platform data visualization library built on NumPy arrays and designed to work with the broader SciPy stack. It was introduced by John Hunter in 2002.

Types of Matplotlib

Matplotlib comes with a wide variety of plots. Plots help to understand trends, and patterns, and to make correlations. They're typically instruments for reasoning about quantitative information.

Some of the sample plots are covered here.

Matplotlib **Line Plot**

Matplotlib **Bar Plot**

Matplotlib **Histograms**

PlotMatplotlib **Scatter**

Plot Matplotlib **Pie Charts**

Matplotlib **Area Plot**

5.2.1 Matplotlib Line Plot

By importing the matplotlib module, defines x and y values for a plotPython, plots the data using the plot() function and it helps to display the plot by using the show() function . The plot() creates a line plot by connecting the points defined by x and y values.

Eg:-

```
# importing matplotlib module

from matplotlib import pyplot as plt

# x-axis values

x = [5, 2, 9, 4, 7]

# Y-axis values

y = [10, 5, 8, 4, 2]

# Function to plot

plt.plot(x, y)

# function to show the plot

plt.show()
```

5.2.2 Matplotlib Bar Plot

```
# importing matplotlib module

from matplotlib import pyplot as plt

# x-axis valuesx = [5, 2, 9, 4, 7]
```

```
# Y-axis values

y = [10, 5, 8, 4, 2]

# Function to plot the bar

plt.bar(x, y)

# function to show the plot

plt.show()
```

5.2.3 Matplotlib Histograms Plot

```
# importing matplotlib module
from matplotlib import pyplot as plt

# Y-axis values
y = [10, 5, 8, 4, 2]

# Function to plot histogram
plt.hist(y)

# Function to show the plot
plt.show()
```

5.2.4 Matplotlib Scatter Plot

```
# importing matplotlib module
from matplotlib import pyplot as plt

# x-axis values
x = [5, 2, 9, 4, 7]

# Y-axis values
y = [10, 5, 8, 4, 2]

# Function to plot scatter
plt.scatter(x, y)

# function to show the plot
plt.show()
```

5.2.5 Matplotlib Pie Charts

```
import matplotlib.pyplot as plt

# Data for the pie chart
labels = ['Geeks 1', 'Geeks 2', 'Geeks 3']
sizes = [35, 35, 30]

# Plotting the pie chart
```

```
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
plt.title('Pie Chart Example')
plt.show()
```

5.2.6 Matplotlib Area Plot

```
import matplotlib.pyplot as plt
# Data
x = [1, 2, 3, 4, 5]
y1, y2 = [10, 20, 15, 25, 30], [5, 15, 10, 20, 25]

# Area Chart
plt.fill_between(x, y1, y2, color='skyblue', alpha=0.4, label='Area 1-2')
plt.plot(x, y1, label='Line 1', marker='o')
plt.plot(x, y2, label='Line 2', marker='o')

# Labels and Title
plt.xlabel('X-axis'),
plt.ylabel('Y-axis'),
plt.title('Area Chart Example')

# Legend and Display
plt.legend(),
plt.show()
```

Lecture 27. Python NumPy

5.3 Numpy:-

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object and tools for working with these arrays. It is the fundamental package for scientific computing with Python. It is open- source software.

5.3.1 Features of NumPy

NumPy has various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Install Python

NumPypip install

numpy

Create a NumPy ndarray Object

NumPy is used to work with arrays. The array object in NumPy is called ndarray.We can create a NumPy ndarray object by using the array() function.

Eg:-

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(type(arr))
```

To create an ndarray, we can pass a list, tuple or any array-like object into the array() method, and it will be converted into an ndarray:

Dimensions in Arrays

NumPy arrays can have multiple dimensions, allowing users to store data in multilayered structures.

Dimensionalities of array:

Name	Example
0D (zero-dimensional)	Scalar – A single element
1D (one-dimensional)	Vector- A list of integers.
2D (two-dimensional)	Matrix- A spreadsheet of data
Name	**Example**
3D (three-dimensional)	Tensor- Storing a color image

2-D Arrays

An array that has 1-D arrays as its elements is called a 2-D array.

These are often used to represent matrix or 2nd order tensors.

Example:-

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)
```

3-D arrays

Example:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(arr)
```

Lecture 28. Python Pandas

5.4 Introduction to Pandas

Pandas is a name from "Panel Data" and is a Python library used for data manipulation and analysis.
Pandas provides a convenient way to analyze and clean data. The Pandas library introduces two new data structuresto Python - Series and DataFrame, both of which are built on top of NumPy.

Why Use Pandas?

1. Handle Large Data Efficiently
2. Tabular Data Representation
3. Data Cleaning and Preprocessing
4. Free and Open-Source

Install Pandas

To install pandas, you need **Python and PIP installed** in your system.

If you have Python and PIP installed already, you can install pandas by entering the following command in the terminal:
pip install pandas

Import Pandas in Python We can import Pandas in Python using the import statement.
import pandas as pd

What can you do using Pandas?

Pandas are generally used for data science but have you wondered why? This is because pandas are used in conjunction with other libraries that are used for data science. It is built on the top of the NumPy library which means that a lot of structures of NumPy are used or replicated in Pandas. The data produced by Pandas are often used as input for plotting functions of Matplotlib, statistical analysis in SciPy, and machine learning algorithms in Scikit-learn. Here is a list of things that we can do using Pandas.

- Data set cleaning, merging, and joining.
- Easy handling of missing data (represented as NaN) in floating point as well as non-floating point data.
- Columns can be inserted and deleted from DataFrame and higher dimensional objects.
- Powerful group by functionality for performing split-apply-combine operations on data sets.
- Data Visulaization

Pandas Data Structures

Pandas generally provide two data structures for manipulating data, They are:

- Series
- DataFrame

Pandas Series

A Pandas Series is a one-dimensional labeled array capable of holding data of any type (integer, string, float, python objects, etc.). The axis labels are collectively called indexes.
Pandas Series is nothing but a column in an Excel sheet. Labels need not be unique but must be a hashable type. The object supports both integer and label-based indexing and provides a host of methods for performing operations involving the index.

Example 1:

```
# import pandas as pd
import pandas as pd

# simple array
data = [1, 2, 3, 4]

ser = pd.Series(data)
print(ser)
```

Example 2:-

```
# import pandas as pd
import pandas as pd

# import numpy as np
import numpy as np

# simple array
data = np.array(['g','e','e','k','s'])

ser = pd.Series(data)
print(ser)
```

DataFrame

Pandas DataFrame is a two-dimensional data structure with labeled axes (rows and columns).

Creating Data Frame

In the real world, a Pandas DataFrame will be created by loading the datasets from existing storage, storage can be SQL Database, CSV file, or an Excel file. Pandas DataFrame can be created from lists, dictionaries, and from a list ofdictionaries, etc.

Example:-

```
import pandas as pd

# Calling DataFrame constructor
df = pd.DataFrame()
print(df)

# list of strings
lst = ['Geeks', 'For', 'Geeks', 'is', 'portal', 'for', 'Geeks']

# Calling DataFrame constructor on
listdf = pd.DataFrame(lst)
print(df)
```

5.5 Hello World Window:

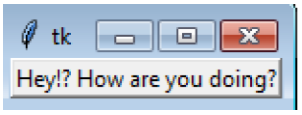
```
import tkinter as tk

# Create the main window
root = tk.Tk()
root.title("Hello World")

# Create a label widget with "Hello, World!" text
label = tk.Label(root, text="Hello, World!")
label.pack()

# Run the main event loop
root.mainloop()
```

Q:- Python Tkinter Program to show onclick event on button

Source Code:

```
import Tkinter as tk
root=tk.Tk()
def click():
    a=e.get()
    a1="Hello "+a
    l1.config(text=a1
    )
l=tk.Label(root,text="name",fg="red",bg="lightgreen")
l.grid(row=0,column=0)
l1=tk.Label(root)
l1.grid(row=1,column=1)
e=tk.Entry(root)
e.grid(row=0,column=1)
b=tk.Button(root,text="click",fg="red",bg="blue",command=click)
b.grid(row=1,column=0)
#button =tk.Button(root, text="Quit", command=root.destroy)

root.mainloop()
```

OUTPUT



Lecture 30. Tkinter and Python Programming

5.6 Tkinter

Python provides various options for developing graphical user interfaces (GUIs). Most important are listed below.

- **Tkinter**− Tkinter is the Python interface to the Tk GUI toolkit shipped with Python. We would look this option in this chapter.
- **wxPython**− This is an open-source Python interface for wxWindow shttp://wxpython.org.
- **JPython**− JPython is a Python port for Java which gives Python scripts seamless access to Java class libraries on the local machine http://www.jython.org

Tkinter Programming

Tkinter is the standard GUI library for Python. Python when combined with Tkinter provides a fast and easy way to create GUI applications. Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit.

Creating a GUI application using Tkinter is an easy task. All you need to do is perform the following steps .

- Import the Tkinter module.
- Create the GUI application main window.
- Add one or more of the above-mentioned widgets to the GUI application.
- Enter the main event loop to take action against each event triggered by the user.

Tk provides the following widgets:

- button
- canvas
- checkbutton
- combobox
- entry
- frame
- label
- labelframe
- listbox
- menu
- menubutton
- message
- notebook
- tk_optionMenu
- progressbar
- radiobutton
- scale
- scrollbar
- separator
- sizegrip
- spinbox
- text
- treeview

Geometry Management

5.7 Tk Widget- All Tkinter widgets have access to specific geometry management methods, which have the purpose of organizing widgets throughout the parent widget area. Tkinter exposes the following geometry manager classes: pack, grid, andplace.

- The pack() Method− This geometry manager organizes widgets in blocks before placing them in the parent widget.
- The grid() Method − This geometry manager organizes widgets in a table-like structure in the parent widget.
- The place() Method − This geometry manager organizes widgets by placing them in a specific position in the parent widget (absolute positions).

WIDGETS

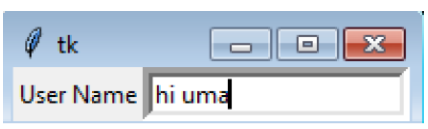
Label

- from tkinter import *
- root = Tk()
- var = StringVar()
- label = Label(root, textvariable = var, relief = RAISED)
- var.set("Hey!? How are you doing?")
- label.pack()
- root.mainloop()
- **OUTPUT**

ENTRY

- from tkinter import *
- top = Tk()
- L1 = Label(top, text = "User Name")
- L1.pack(side = LEFT)
- E1 = Entry(top, bd = 5)
- E1.pack(side = RIGHT)
- top.mainloop()

Output:

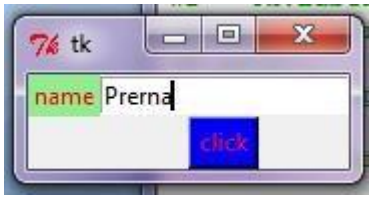


button

- **# Code to demonstrate a button**
- import Tkinter as tk

- root=tk.Tk()
- l=tk.Label(root,text="name",fg="red",bg="lightgreen")
- l.grid(row=0,column=0)
- e=tk.Entry(root)
- e.grid(row=0,column=1)
- b=tk.Button(root,text="click",fg="red",bg="blue")
- b.grid(row=1,column=1)
- root.mainloop()

OUTPUT

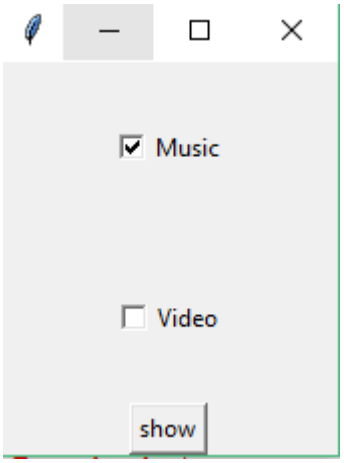


-Lecture 32. Tkinter example

CheckButton

```
from tkinter import *
import tkinter
top = Tk()
CheckVar1 =
IntVar()CheckVar2
= IntVar()def
show():
   print(CheckVar1.get()
   )
   print(CheckVar2.get()
   )

C1 = Checkbutton(top, text = "Music", variable = CheckVar1,
            \onvalue = 5, offvalue = 0, height=5, \
            width = 20, )
C2 = Checkbutton(top, text = "Video", variable = CheckVar2,
            \onvalue = 6, offvalue = 0, height=5, \
            width = 20)
B1=Button(top,text="show",command=show)

C1.pack()
C2.pack()
B1.pack()
top.mainloop()
```

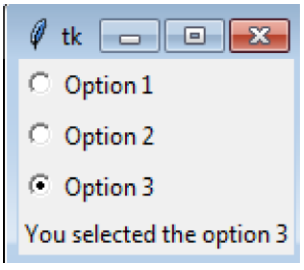
OUTPUT



Radiobutton (int as well as string)

```
from tkinter import *
def sel():
   selection = "You selected the option " + str(var.get())
   label.config(text = selection)
root = Tk()
var = IntVar()
R1 = Radiobutton(root, text = "Option 1", variable = var, value = 1,command =
sel)R1.pack( )
R2 = Radiobutton(root, text = "Option 2", variable = var, value = 2, command =
sel)R2.pack( )
R3 = Radiobutton(root, text = "Option 3", variable = var, value = 3, command =
sel)R3.pack( )
label =
Label(root)
label.pack()
root.mainloop()
```

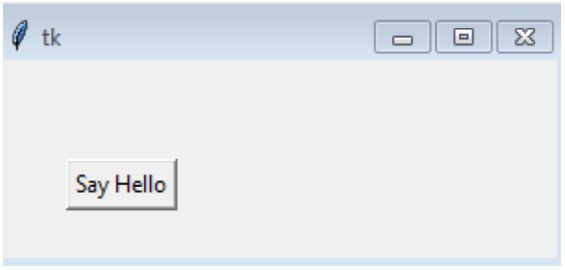
OUTPUT



MessageBox (Showinfo)

```
from tkinter import *
from tkinter import messagebox
top = Tk()
top.geometry("300x100")
def hello():
   messagebox.showinfo("Say Hello", "Hello World")

B1 = Button(top, text = "Say Hello", command = hello)
B1.place(x = 35,y = 50)
top.mainloop()
```

OUTPUT:-



Frame

```
from tkinter import *
root = Tk()
frame = Frame(root)
frame.pack()
bottomframe = Frame(root)
bottomframe.pack( side = BOTTOM )
redbutton = Button(frame, text = "Red", fg = "red")
redbutton.pack( side = LEFT)
greenbutton = Button(frame, text = "Brown", fg="brown")
greenbutton.pack( side = LEFT )
bluebutton = Button(frame, text = "Blue", fg = "blue")
bluebutton.pack( side = LEFT )
blackbutton = Button(bottomframe, text = "Black", fg = "black")
blackbutton.pack( side = BOTTOM)
root.mainloop()
```

OUTPUT:-

Lecture 33. Python Programming with IDE

5.8 Python Programming with IDE- An integrated development environment (IDE) refers to a software application that offers computer programmers with extensive software development abilities. IDEs most often consist of a source code editor, build automation tools, and a debugger. Most modern IDEs have intelligent code completion. In this article, you will discover the best Python IDEs currently available and present in the market.

What Is an IDE?

- An IDE enables programmers to combine the different aspects of writing a computer program.
- IDEs increase programmer productivity by introducing features like editing source code, building executables, and debugging.

IDE vs. Code Editor: What's the Difference?

- An Integrated Development Environment (IDE) is a software application that provides tools and resources to help developers write and debug code. An IDE typically includes

  A source code editor

  A compiler or interpreter

  An integrated debugger

  A graphical user interface (GUI)

- A code editor is a text editor program designed specifically for editing source code. It typically includes features that help in code development, such as syntax highlighting, code completion, and debugging.
- The main difference between an IDE and a code editor is that an IDE has a graphical user interface (GUI) while a code editor does not. An IDE also has features such as code completion, syntax highlighting, and debugging, which are not found in a code editor.
- Code editors are generally simpler than IDEs, as they do not include many other IDE components. As such, code editors are typically used by experienced developers who prefer to configure their development environment manually.

Top Python IDEs

i. IDLE

ii. PyCharm

iii. Visual Studio Code

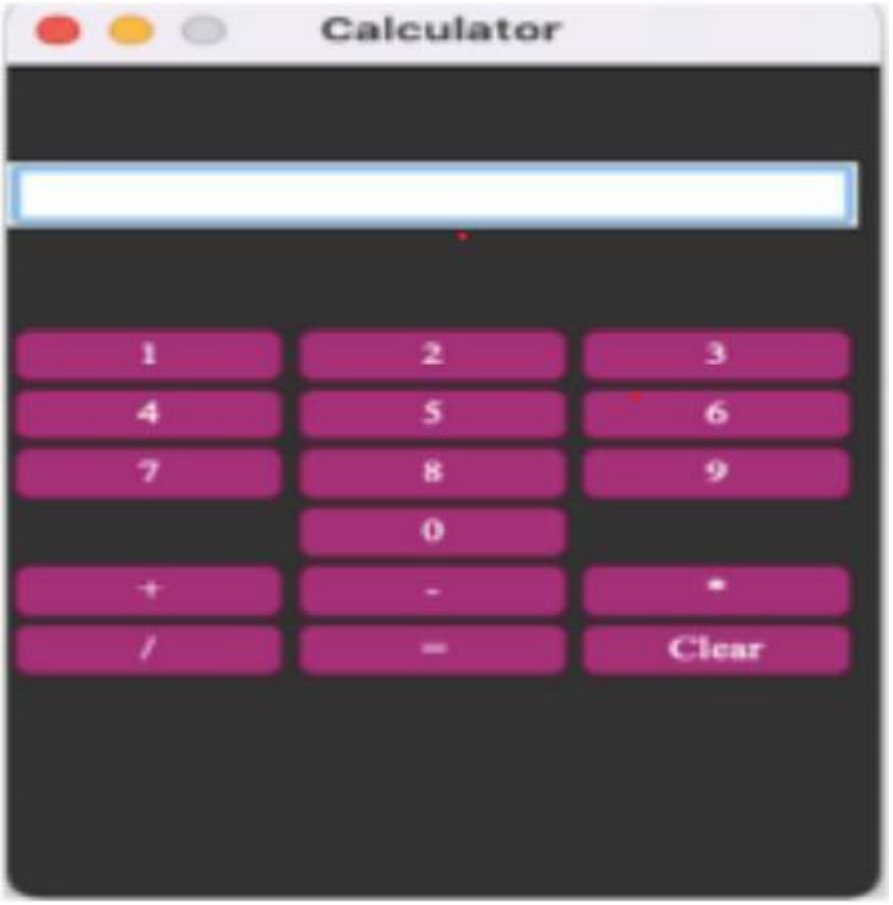
iv. Jupyter

Important Questions CO-5

i. What is the significance of module in Python? What are the methods of importing a module? Explain with

suitable example (2021-22)

ii. Design a calculator with the following buttons and functionalities like addition, subtraction, multiplication, division and clear. (2023-24 odd sem)



iii. Construct a program to read cities.csv dataset, remove last column and save it in an array. Save the last column to another array. Plot the first two columns.(2023-24 odd sem)

iv. Construct a plot for following dataset using matplotlib : .(2023-24 odd sem)

Food	Calories	Potassium	fat
Meat	250	40	8
Banana	130	55	5
Avocados	140	20	3
Sweet Potatoes	120	30	6
Spinach	20	40	1
Watermelon	20	32	1.5
Coconut water	10	10	0
Beans	50	26	2
Legumes	40	25	1.5
Tomato	19	20	2.5

v. Describe how to generate random numbers using NumPy. Write a python program to create an array of 5 random integers between 10 and 50. .(2023-24 even sem)

Lecture 34-40 Programming Questions

i. Python Program to exchange the values of two numbers without using a temporary variable.
ii. Python Program to take the temperature in Celsius and convert it to Fahrenheit.
iii. Python Program to read two numbers and print their quotient and remainder.
iv. Python Program to find the area of a triangle given all three sides.
v. Python Program to read height in centimeters and then convert the height to feet and inches
vi. Python Program to compute simple interest given all the required values.
vii. Python Program to check whether a given year is a leap year or not.
viii. Python Program to take in the marks of 5 subjects and display the grade.
ix. Python Program to check if a number is an Armstrong number.
x. Python Program to find the sum of digits in a number.
xi. Python Program to print odd numbers within a given range.
xii. Python Program to check whether a given number is a palindrome.
xiii. Python Program to print all numbers in a range divisible by a given number.
xiv. Python Program to read a number n and print an inverted star pattern of the desired size.
xv. Python Program to find the sum of first N Natural Numbers.
xvi. Python Program to read a number n and print and compute the series "1+2+…+n=".
xvii. Python Program to find the sum of series: 1 + 1/2 + 1/3 + ….. + 1/N.
xviii. Python Program to find the sum of series: 1 + x^2/2 + x^3/3 + … x^n/n.
xix. Python Program to find the sum of series: 1 + 1/2 + 1/3 + ….. + 1/N.
xx. Python program to find whether a number is a power of two.
xxi. Python Program to find the second largest number in a list.
xxii. Python Program to put the even and odd elements in a list into two different lists.
xxiii. Python Program to merge two lists and sort it.
xxiv. Python Program to find the second largest number in a list using bubble sort.
xxv. Python Program to find the intersection of two lists.
xxvi. Python Program to sort a list of tuples in increasing order by the last element in each tuple.
xxvii. Python Program to remove the duplicate items from a list.
xxviii. Python Program to replace all occurrences of 'a' with '$' in a string.
xxix. Python Program to detect if two strings are anagrams.
xxx. Python Program to count the number of vowels in a string.
xxxi. Python Program to calculate the length of a string without using library functions.
xxxii. Python Program to check if a string is a palindrome or not.
xxxiii. Python Program to check if a substring is present in a given string.
xxxiv. Python Program to add a key-value pair to a dictionary.
xxxv. Python Program to concatenate two dictionaries into one dictionary.
xxxvi. Python Program to check if a given key exists in a dictionary or not.
xxxvii. Python Program to remove the given key from a dictionary.
xxxviii. Python Program to count the frequency of words appearing in a string using a dictionary.
xxxix. Python Program to create a dictionary with key as first character and value as words starting with that character.
xl. Python Program to count the number of vowels present in a string using sets
xli. Python Program to check common letters in the two input strings.
xlii. Python Program to display which letters are present in both the strings.
xliii. Python Program to find the fibonacci series using recursion.
xliv. Python Program to find the factorial of a number using recursion.
xlv. Python Program to find the GCD of two numbers using recursion.
xlvi. Python Program to reverse a string using recursion.
xlvii. Python Program to read the contents of a file.
xlviii. Python Program to count the number of words in a text file.
xlix. Python Program to copy the contents of one file into another.
l. Python Program to append the contents of one file to another file.
li. Python Program to read a file and capitalize the first letter of every word in the file